

DOKUMEN CD - 4



**Pengembangan Computer Vision pada Robot TIFA untuk
Deteksi Kebersihan Meja Pelanggan di TelU Coffee**

Oleh :

Ariski Myardi 1103220019

Dwi Agus Cahyo Ilahi 1103223043

Gita Adi Pangestu 1103223225

**PRODI S1 TEKNIK KOMPUTER
FAKULTAS TEKNIK ELEKTRO
UNIVERSITAS TELKOM
BANDUNG
2026**

Dokumentasi Produk Capstone Design

Lembar Pengesahan Dokumen

Judul Capstone Design : Pengembangan Computer Vision pada Robot TIFA untuk Deteksi Kondisi Meja Pelanggan di TelU Coffee

Jenis Dokumen : Implementasi

Nomor Dokumen : FTE-CD-4

Nomor Revisi : 9

Tanggal Pengesahan : 21/05/2026

Fakultas : Fakultas Teknik Elektro

Program Studi : S1 Teknik Komputer

Jumlah Halaman : 51

Data Pemeriksaan dan Persetujuan			
Ditulis Oleh	Nama : Ariski Myardi	Jabatan : Mahasiswa	
	NIM : 11032230019	Tanda Tangan	
	Nama : Dwi Agus Cahyo Ilahi	Jabatan : Mahasiswa	
	NIM : 1103223043	Tanda Tangan	
	Nama : Gita Adi Pangestu	Jabatan : Mahasiswa	
	NIM : 1103223225	Tanda Tangan	
Disetujui Oleh	Nama : Dr. Budhi Irawan, S.Si., M.T.	Jabatan : Pembimbing 1	
	Tanggal :	Tanda Tangan	
	Nama : Faisal Candrasyah Hasibuan, S.T., M.T	Jabatan : Pembimbing 2	
	Tanggal : 21 Mei 2026	Tanda Tangan	

Timeline Revisi Dokumen

No	Tanggal Perbaikan	Bagian / Halaman	Catatan Revisi (Poin Perbaikan)	Detail Perbaikan yang Dilakukan
1	02 April 2026	Seluruh Dokumen	Pemberian dan Penyesuaian Sub-bab	Menambahkan dan merapikan hierarki sub-bab pada keseluruhan dokumen agar lebih terstruktur.
2	08 April 2026	Seluruh Dokumen	Perbaikan Penulisan Akademik (CD-4)	Memperbaiki tata letak <i>caption</i> gambar/kode, urutan tabel, dan margin penulisan sesuai standar <i>template</i> Capstone Design (CD-4).
3	15 April 2026	Bab 4	Revisi Struktur Bab 4	Merombak struktur Bab 4 dengan menambahkan sub-bab khusus untuk implementasi <i>hardware</i> , <i>software</i> , prosedur, dan arsitektur.

4	21 April 2026	Sub-bab 4.1.1	Penambahan Implementasi <i>Hardware</i>	Menambahkan spesifikasi sensor Kinect, Raspberry Pi 4, modul <i>step-up</i> , jaringan komunikasi, serta Tabel Alat dan Bahan.
5	28 April 2026	Sub-bab 4.1.2	Penambahan Implementasi <i>Software</i>	Menjabarkan peran bahasa Python, ROS 2, OpenCV, PyTorch, YOLOv8, dan mekanisme protokol UDP.
6	04 Mei 2026	Sub-bab 4.2.3	Penambahan <i>Mapping</i> Koordinat Meja	Menambahkan tabel pemetaan koordinat spasial meja beserta narasi mekanisme robot memvalidasi posisi meja target.
7	10 Mei 2026	Sub-bab 4.3	Penambahan Pengujian Sistem	Memasukkan dokumentasi hasil akurasi model, metrik FPS, latensi UDP, <i>script</i> penerima UDP, dan <i>node</i> ROS

				2.
8	16 Mei 2026	Sub-bab 4.3.1.4	Penambahan Analisis Hasil	Menambahkan analisis teknis mendalam terkait presisi, <i>recall</i> , FPS, latensi, serta justifikasi pemilihan YOLOv8n.
9	21 Mei 2026	Lampiran A	Analisis Dataset Dan modul Ai	Menambahkan anslisi dataset dan juga modul AI

4.1 Deskripsi Umum Implementasi

Implementasi pada penelitian ini berfokus pada pengembangan sistem pemantauan visual cerdas yang terintegrasi dengan robot Tel-U Interactive Food Assistant (TIFA) untuk mendukung optimalisasi operasional layanan di Tel-U Coffee. Sistem yang dikembangkan merupakan kombinasi antara perangkat keras dan perangkat lunak yang bekerja secara terintegrasi untuk melakukan proses deteksi kondisi meja secara otomatis menggunakan pendekatan *computer vision*. Pengembangan sistem ini dilatarbelakangi oleh kondisi operasional Tel-U Coffee, di mana proses identifikasi meja yang telah selesai digunakan pelanggan masih dilakukan secara manual oleh staf. Pada kondisi tertentu, khususnya saat jam operasional padat (*peak hours*), staf mengalami kesulitan dalam memantau meja yang perlu segera dibersihkan. Hal tersebut menyebabkan keterlambatan proses pembersihan meja yang berdampak pada menurunnya efisiensi layanan dan kenyamanan pelanggan berikutnya.

Untuk menjawab permasalahan tersebut, sistem ini dikembangkan sebagai modul pendukung pada robot TIFA agar mampu melakukan pengamatan kondisi lingkungan secara mandiri. Sebelumnya, robot TIFA telah digunakan sebagai robot layanan untuk membantu proses pengantaran makanan dan minuman. Namun, sistem tersebut belum memiliki kemampuan untuk melakukan analisis visual terhadap kondisi meja di area operasional. Oleh karena itu, implementasi pada penelitian ini difokuskan pada penambahan subsistem kecerdasan buatan yang memungkinkan robot mengenali kondisi meja secara otomatis. Sistem yang dibangun menggunakan arsitektur berbasis *edge acquisition* dan *centralized inference*. Proses pengambilan data visual dilakukan menggunakan perangkat kamera yang terpasang pada robot, kemudian citra yang diperoleh dikirimkan untuk diproses menggunakan model kecerdasan buatan berbasis YOLOv8. Model tersebut digunakan untuk melakukan deteksi objek dan klasifikasi kondisi meja ke dalam beberapa kategori, yaitu *meja bersih*, *meja kotor*, dan *meja terisi*.

Hasil deteksi dari model kecerdasan buatan selanjutnya diteruskan melalui jaringan komunikasi untuk keperluan pemantauan sistem secara terintegrasi. Dengan mekanisme tersebut, robot dapat memberikan informasi mengenai kondisi meja yang memerlukan penanganan tanpa harus bergantung sepenuhnya pada pengamatan manual oleh staf. Pada implementasinya, pengembangan sistem ini dibatasi pada subsistem deteksi visual dan

klasifikasi kondisi meja sebagai bagian dari pengembangan kemampuan robot TIFA. Sistem navigasi utama, aktuator pergerakan robot, serta mekanisme pengendalian robot yang telah tersedia sebelumnya tidak menjadi bagian dari ruang lingkup implementasi penelitian ini.

4.1.1 Hardware

A. Kinect Xbox 360

Sensor utama yang digunakan pada sistem Computer Vision Robot TIFA adalah Kinect Xbox 360. Pada implementasi ini, Kinect digunakan sebagai perangkat akuisisi citra RGB untuk menangkap kondisi meja pelanggan secara real-time pada area operasional TelU Coffee. Berbeda dengan implementasi Kinect pada umumnya yang memanfaatkan fitur *depth sensing* dan *skeletal tracking*, pada penelitian ini Kinect hanya digunakan sebagai kamera RGB. Pemanfaatan Kinect sebagai kamera RGB dipilih karena memiliki kualitas citra yang cukup baik untuk kebutuhan deteksi objek menggunakan metode YOLOv8. Sensor Kinect dihubungkan menuju Raspberry Pi 4 melalui antarmuka USB. Data citra RGB yang diperoleh kemudian diteruskan menuju Processing PC melalui jaringan lokal untuk diproses pada tahap inferensi model AI.



Gambar 4.1 Sensor Kinect Xbox 360 yang digunakan sebagai akuisisi citra RGB pada Robot TIFA

B. Raspberry Pi 4

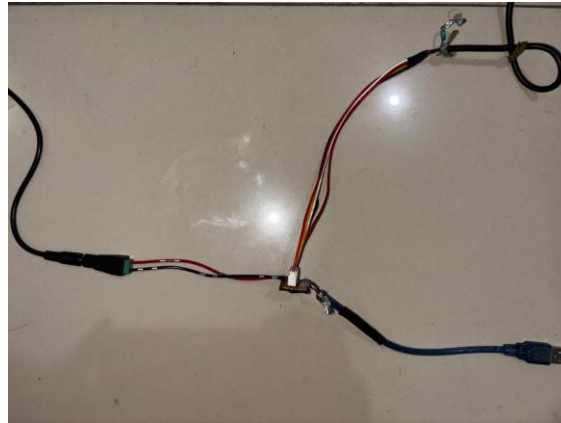
Raspberry Pi 4 Model B (RAM 4 GB) digunakan sebagai perangkat *edge acquisition* dan UDP forwarder. Raspberry Pi tidak menjalankan inferensi YOLOv8 secara langsung untuk mengurangi beban komputasi, mengingat proses inferensi model *deep learning* membutuhkan sumber daya pemrosesan yang cukup tinggi. Raspberry Pi 4 menjalankan sistem operasi Raspberry Pi OS dan diakses secara headless menggunakan komunikasi Secure Shell (SSH) dari Processing PC. Foto fisik perangkat Raspberry Pi 4 Model B yang digunakan beserta konfigurasi kabel data dan dongle WiFi dapat dilihat pada Gambar 4.2.



Gambar 4.2 Raspberry Pi 4 Model B sebagai perangkat *edge acquisition* dan UDP forwarder

C. Modul Step-Up DC-to-DC Converter

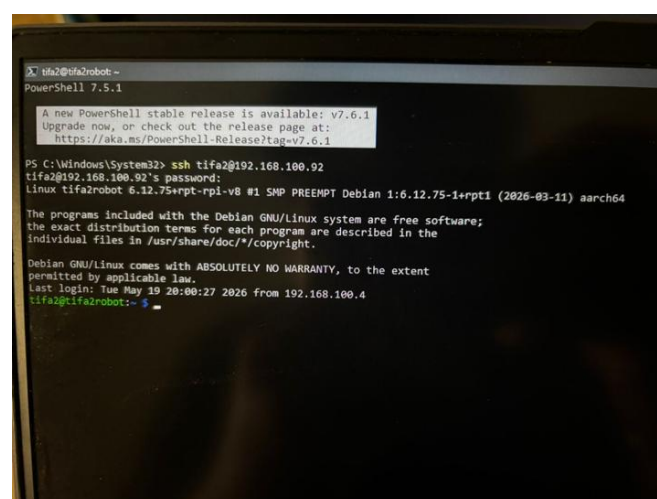
Modul step-up DC-to-DC converter (XL6009 atau sejenis) digunakan untuk meningkatkan tegangan keluaran dari 5V menjadi 12V sebagai sumber daya utama sensor Kinect Xbox 360. Penggunaan modul ini memungkinkan Kinect dapat beroperasi secara portabel pada platform Robot TIFA tanpa memerlukan adaptor daya eksternal bawaan Kinect. Selain itu, penggunaan step-up converter membantu menjaga kestabilan suplai daya selama sistem berjalan sehingga proses akuisisi citra dapat berlangsung secara kontinu.



Gambar 4.3 Rangkaian perkabelan Kinect Xbox 360 dan modul step-up converter

D. Jaringan dan Komunikasi Sistem

Komunikasi data antara Processing PC, Raspberry Pi 4, dan subsistem navigasi Robot TIFA dilakukan menggunakan jaringan lokal berbasis wireless hotspot. Seluruh perangkat berada dalam subnet 192.168.100.0/24. Processing PC bertindak sebagai pemilik kamera dan model AI yang mengirim sinyal STATUS:KOTOR via UDP ke Raspberry Pi 4. Raspberry Pi 4 kemudian meneruskan sinyal tersebut ke modul navigasi robot melalui ROS 2 atau payload JSON, tergantung kebutuhan integrasi. Akses Raspberry Pi dari Processing PC dilakukan melalui Secure Shell (SSH). Tampilan sesi SSH dari Processing PC (PowerShell) menuju Raspberry Pi (tifa2@tifa2robot) dapat dilihat pada Gambar 4.3.



Gambar 4.4 Sesi SSH dari Processing PC (Windows PowerShell) menuju Raspberry Pi 4 (192.168.100.92)

4.1.2 Software

Implementasi perangkat lunak pada sistem Computer Vision Robot TIFA dikembangkan menggunakan beberapa framework dan library utama yang saling terintegrasi untuk mendukung proses akuisisi citra, inferensi kecerdasan buatan, komunikasi data, serta integrasi navigasi robot. Perangkat lunak utama yang digunakan meliputi Python 3.10, ROS 2, OpenCV, Ultralytics YOLOv8, dan PyTorch. Seluruh perangkat lunak tersebut dijalankan pada lingkungan Processing PC berbasis Windows 11 dan Raspberry Pi OS.

4.1.2.1 Python

Python 3.10 digunakan sebagai bahasa pemrograman utama dalam pengembangan seluruh sistem Computer Vision Robot TIFA. Pemilihan Python dilakukan karena memiliki kompatibilitas tinggi terhadap berbagai library computer vision, *deep learning*, dan robotika. Pada implementasi ini, Python digunakan untuk menjalankan proses akuisisi citra Kinect, inferensi YOLOv8, pengiriman data UDP, pengolahan payload JSON, serta komunikasi antar node pada ROS 2.

4.1.2.2 ROS 2 (Robot Operating System 2)

ROS 2 digunakan sebagai framework komunikasi antar subsistem robot menggunakan arsitektur publisher–subscriber. Pada implementasi Robot TIFA, ROS 2 dimanfaatkan untuk mendistribusikan data hasil deteksi meja menuju subsistem navigasi robot secara real-time. Penggunaan ROS 2 memungkinkan sistem menjadi modular, sehingga subsistem computer vision dan subsistem navigasi dapat dikembangkan secara terpisah tanpa saling bergantung secara langsung.

4.1.2.3 OpenCV

OpenCV digunakan sebagai library utama pengolahan citra digital pada sistem. OpenCV bertugas membaca stream kamera dari Kinect Xbox 360, melakukan konversi format citra, menampilkan hasil inferensi secara visual, serta menggambar bounding box dan *confidence score* hasil deteksi YOLOv8 pada frame video secara real-time. Selain itu, OpenCV juga digunakan untuk mengatur resolusi input kamera agar sesuai dengan kebutuhan model inferensi.

4.1.2.4 Ultralytics YOLOv8

Ultralytics YOLOv8 digunakan sebagai framework utama implementasi model *deep learning* untuk proses deteksi dan klasifikasi kondisi meja pelanggan. Model YOLOv8 yang

digunakan telah dilatih menggunakan dataset kondisi meja pada lingkungan TelU Coffee dan mampu mengklasifikasikan kondisi meja ke dalam tiga kategori utama, yaitu meja bersih (*clean*), meja terisi pelanggan (*occupied*), dan meja kotor (*dirty*). *Framework Ultralytics* dipilih karena memiliki performa inferensi real-time yang baik serta integrasi yang sederhana dengan bahasa pemrograman Python.

4.1.3 Prosedur Implementasi

Proses implementasi sistem dilakukan melalui beberapa tahapan utama sebagai berikut:

1. **Pemilihan Komponen.** Tahap awal dilakukan dengan menentukan perangkat keras yang digunakan, yaitu Raspberry Pi 4 sebagai perangkat *edge acquisition* / forwarder dan Kinect Xbox 360 sebagai sensor akuisisi citra RGB.
2. **Pengembangan Dataset dan Model.** Dataset kondisi meja pelanggan dikumpulkan dari area TelU Coffee dengan total sekitar 3.000 citra. Selanjutnya dilakukan proses anotasi (labeling) untuk setiap kategori kondisi meja menggunakan platform Roboflow sebelum digunakan pada proses pelatihan model YOLOv8.
3. **Perakitan Hardware dan Sistem Kelistrikan.** Tahap ini meliputi integrasi Raspberry Pi 4, Kinect Xbox 360, modul step-up converter, serta pengaturan distribusi daya pada sistem robot.
4. **Implementasi dan Integrasi Komunikasi.** Pada tahap ini dikembangkan skrip Python yang bertugas membaca stream kamera, menjalankan inferensi YOLOv8, melakukan klasifikasi kondisi meja, mengirim sinyal UDP ke Raspberry Pi, dan menerbitkan pesan ROS 2 ke subsistem navigasi robot.
5. **Pengujian Sistem.** Pengujian dilakukan untuk memastikan sistem mampu mendeteksi kondisi meja pelanggan secara real-time serta mengirimkan data hasil deteksi dengan stabil menuju sistem navigasi robot.

4.1.4 Alat dan Bahan yang Digunakan

Tabel 4.1 Alat dan Bahan

No	Komponen	Spesifikasi / Keterangan	Jumlah
1	Kinect Xbox 360	Sensor kamera RGB resolusi 640×480, antarmuka USB 2.0, digunakan sebagai perangkat akuisisi citra real-time pada Robot TIFA.	1 unit

No	Komponen	Spesifikasi / Keterangan	Jumlah
2	Raspberry Pi 4 Model B	Komputer mini berbasis ARM Cortex-A72 quad-core 1,5 GHz, RAM 4 GB, sistem operasi Raspberry Pi OS. Berfungsi sebagai edge acquisition device dan UDP forwarder.	1 unit
3	MicroSD Card	Kapasitas 32 GB, Class 10, digunakan sebagai media penyimpanan sistem operasi Raspberry Pi 4.	1 unit
4	Processing PC / Laptop	Laptop berbasis Windows 11 dengan dukungan CUDA-capable GPU NVIDIA, RAM $\geq$ 8 GB, digunakan untuk training dan inferensi model YOLOv8.	1 buah
5	Kabel USB Kinect	Kabel adaptor USB Type-A bawaan Kinect Xbox 360 untuk komunikasi data ke Processing PC.	1 buah
6	Wireless Router / Hotspot WiFi	Jaringan lokal 2,4 GHz untuk komunikasi UDP antara Processing PC, Raspberry Pi 4, dan modul navigasi Robot TIFA.	1 set
7	Modul Step-Up DC-to-DC Converter	Konverter tegangan 5V $\rightarrow$ 12V (XL6009 / sejenis), arus maksimum 2A, untuk suplai daya sensor Kinect Xbox 360 secara portabel.	1 buah
8	Heatsink Raspberry Pi	Heatsink aluminium dan tembaga untuk menjaga suhu CPU Raspberry Pi 4 selama operasi kontinu.	1 set
9	Power Bank 5V/3A	Sumber daya portabel untuk Raspberry Pi 4 selama uji coba di lapangan.	1 buah
10	Sistem Operasi Raspberry Pi OS	Sistem operasi utama pada Raspberry Pi 4, kernel Linux 6.12.75+rpt-rpi-v8.	Software
11	Sistem Operasi Windows 11	Sistem operasi Processing PC untuk training model dan inferensi YOLOv8 secara real-time.	Software

No	Komponen	Spesifikasi / Keterangan	Jumlah
12	Python 3.10	Bahasa pemrograman utama yang digunakan untuk implementasi seluruh skrip sistem.	Software
13	ROS 2 (Robot Operating System 2)	Framework komunikasi antar node robot dengan arsitektur Publisher–Subscriber, digunakan untuk integrasi dengan subsistem navigasi.	Software
14	OpenCV 4.x	Library pengolahan citra digital, digunakan untuk akuisisi frame Kinect dan rendering bounding box.	Software
15	Ultralytics YOLOv8	Framework deteksi objek berbasis deep learning yang digunakan untuk klasifikasi kondisi meja.	Software
16	Roboflow	Platform berbasis web untuk anotasi, augmentasi, dan manajemen dataset citra meja pelanggan.	Software
17	OpenSSH (SSH Client/Server)	Digunakan untuk remote access Raspberry Pi 4 dari Processing PC melalui PowerShell.	Software
18	Visual Studio Code	Editor source code untuk pengembangan dan debugging seluruh skrip sistem.	Software

Berdasarkan Tabel 4.1, dapat dianalisis bahwa spesifikasi perangkat keras dan lunak dipilih dengan mengutamakan prinsip mobilitas dan efisiensi komputasi (*computation offloading*). Pada sisi perangkat keras, penggunaan Raspberry Pi 4 yang dikombinasikan dengan *Power Bank* dan modul *Step-Up DC-to-DC Converter* memastikan bahwa sistem akuisisi citra (Kinect Xbox 360) dapat beroperasi secara portabel pada badan Robot TIFA tanpa membebani kelistrikan utama robot. Alih-alih memaksakan proses kecerdasan buatan pada *single-board computer* yang memiliki keterbatasan memori dan rentan mengalami *overheating*, beban inferensi berat dialihkan sepenuhnya ke *Processing PC* yang memiliki spesifikasi GPU memadai.

Pada sisi perangkat lunak, pemilihan *stack* teknologi yang berpusat pada Python 3.10 memberikan fleksibilitas tinggi untuk mengintegrasikan proses. *OpenCV* bertugas menangani ekstraksi *frame* dari sensor, *Ultralytics YOLOv8* menangani klasifikasi visual secara *real-time*,

dan protokol UDP serta ROS 2 menjamin transmisi data dari PC kembali ke subsistem navigasi robot. Analisis terhadap kombinasi ini menunjukkan bahwa arsitektur sistem mampu berjalan dengan latensi yang minimal di lingkungan operasional TelU Coffee, karena setiap perangkat menjalankan fungsi yang spesifik sesuai dengan kapasitas komputasinya.

4.2 Detail Implementasi Sistem

Implementasi sistem pada Robot TIFA dirancang untuk mendukung proses deteksi kondisi meja pelanggan secara real-time menggunakan teknologi computer vision berbasis YOLOv8. Sistem terdiri atas tiga subsistem utama, yaitu subsistem akuisisi citra (*edge acquisition*), subsistem pemrosesan kecerdasan buatan (*AI processing*), dan subsistem navigasi robot.

Pada tahap implementasi, sensor Kinect Xbox 360 digunakan sebagai perangkat akuisisi citra RGB untuk menangkap kondisi meja pelanggan pada area operasional TelU Coffee secara real-time. Dalam penelitian ini, fitur *depth sensing* dan *skeletal tracking* pada Kinect tidak dimanfaatkan. Sensor Kinect hanya difungsikan sebagai kamera RGB untuk memperoleh frame citra yang akan diproses oleh sistem deteksi objek.

Raspberry Pi 4 digunakan sebagai perangkat *edge acquisition* dan *communication forwarder*. Perangkat ini bertugas menjaga konektivitas sistem robot, menangani komunikasi data, serta meneruskan informasi hasil deteksi menuju subsistem navigasi robot. Untuk mengurangi beban komputasi pada Raspberry Pi 4, proses inferensi model YOLOv8 tidak dijalankan langsung pada Raspberry Pi, melainkan dipindahkan ke Processing PC eksternal yang memiliki kemampuan komputasi lebih tinggi.

Pada sisi Processing PC, sistem menjalankan proses pembacaan stream kamera dari Kinect Xbox 360 menggunakan OpenCV, dilanjutkan dengan inferensi model YOLOv8 secara real-time. Hasil inferensi kemudian digunakan untuk melakukan klasifikasi kondisi meja ke dalam tiga kategori utama, yaitu meja bersih (*clean*), meja terisi pelanggan (*occupied*), dan meja kotor (*dirty*). Selain melakukan klasifikasi, sistem juga menghasilkan informasi tambahan berupa identitas meja, nilai *confidence score*, serta koordinat posisi meja pada area operasional TelU Coffee.

Setelah proses inferensi selesai dilakukan, hasil deteksi dikirimkan menuju Raspberry Pi 4 menggunakan komunikasi UDP melalui jaringan lokal nirkabel (*wireless local network*).

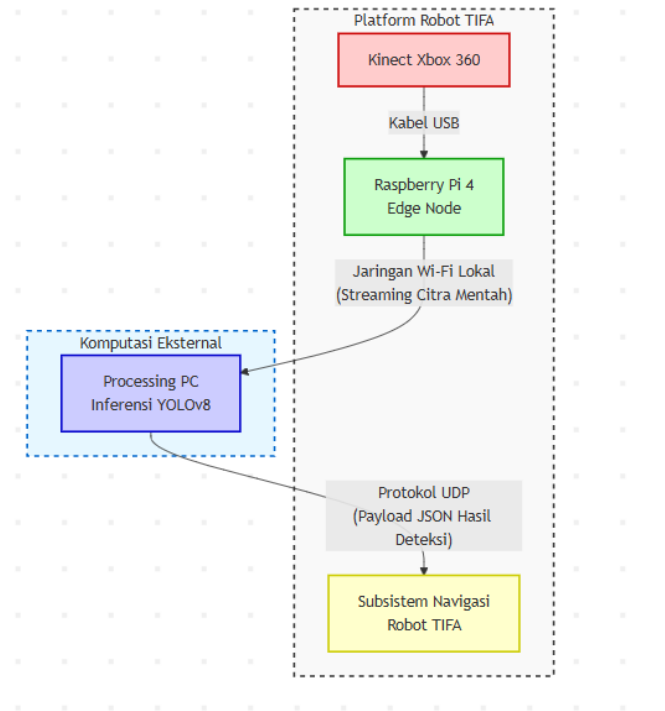
Raspberry Pi kemudian meneruskan informasi tersebut menuju subsistem navigasi robot menggunakan komunikasi ROS 2 maupun *payload* berbasis JSON sesuai kebutuhan integrasi sistem.

Data hasil deteksi yang dikirimkan mencakup identitas meja (*table_id*), status kondisi meja, koordinat posisi meja, *confidence score*, dan *timestamp*. Informasi tersebut digunakan oleh subsistem navigasi robot untuk menentukan target meja dan melakukan proses *path planning* menuju lokasi yang terdeteksi memiliki kondisi meja kotor.

Selain itu, sistem menggunakan pendekatan *predefined coordinate mapping*, yaitu setiap meja pada area TelU Coffee telah memiliki identitas unik beserta koordinat tetap pada peta navigasi robot. Dengan pendekatan ini, robot dapat langsung menentukan lokasi tujuan berdasarkan data hasil deteksi yang diterima dari subsistem computer vision.

4.2.1 Arsitektur Implementasi Sistem

Arsitektur sistem dibangun menggunakan pendekatan komputasi terdistribusi (*computation offloading*) untuk meningkatkan efisiensi. Sensor Kinect Xbox 360 yang terhubung ke Raspberry Pi pada badan robot bertugas menangkap citra RGB secara *real-time*, kemudian melakukan *streaming* data tersebut ke *Processing PC* melalui jaringan *wireless* lokal. Setelah *Processing PC* menjalankan inferensi YOLOv8 untuk mendeteksi kondisi meja pelanggan, hasil deteksi (berupa status meja, *confidence score*, dan identitas meja) langsung dikirimkan ke subsistem navigasi robot menggunakan protokol UDP berupa *payload* JSON atau melalui integrasi ROS 2. Berikut merupakan rancangan umum sistem :



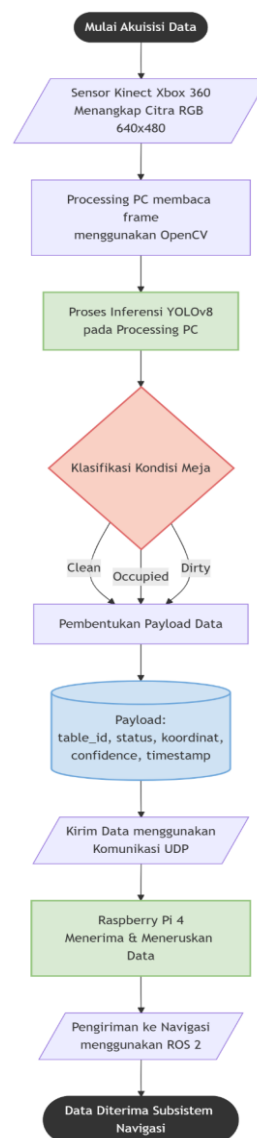
Gambar 4.5 Implementasi sistem *Computer Vision* Robot TIFA

Berdasarkan Gambar 4.5, arsitektur perangkat keras dirancang menggunakan pendekatan *computation offloading*, di mana Raspberry Pi difungsikan sebagai *edge node* untuk akuisisi data sensor Kinect secara langsung di badan robot. Analisis terhadap desain topologi ini menunjukkan efisiensi yang sangat tinggi pada sisi mobilitas robot. Dengan menyambungkan Kinect pada Raspberry Pi, Robot TIFA tidak perlu mengangkut *Processing PC* yang berat dan memakan banyak daya baterai. Citra yang ditangkap oleh Raspberry Pi ditransmisikan secara nirkabel menuju *Processing PC* eksternal untuk dilakukan inferensi YOLOv8. Selanjutnya, PC hanya mengirimkan *payload* berupa data ringan (status kebersihan dan identitas meja) kembali ke subsistem navigasi robot. Pemisahan beban kerja (*workload separation*) ini memastikan bahwa proses AI yang berat tidak mengganggu komputasi pergerakan roda dan navigasi bawaan robot, sekaligus menjaga performa deteksi tetap optimal secara *real-time*.

4.2.2 Aliran Data Sistem

Aliran data sistem diawali dengan akuisisi citra RGB beresolusi 640×480 piksel secara *real-time* menggunakan sensor Kinect Xbox 360. *Frame* citra tersebut dibaca oleh *Processing*

PC menggunakan *library* OpenCV untuk diproses oleh model YOLOv8, yang akan mengklasifikasikan kondisi meja menjadi kategori *clean*, *occupied*, atau *dirty*. Hasil inferensi tersebut kemudian disusun menjadi *payload* data (berisi *table_id*, status meja, koordinat, *confidence score*, dan *timestamp*), lalu dikirimkan melalui komunikasi UDP menuju Raspberry Pi 4 yang bertindak sebagai *forwarder* untuk meneruskan perintah tersebut ke subsistem navigasi robot menggunakan *framework* ROS 2. Berikut diagram aliran data sistem deteksi kondisi meja pelanggan



Gambar 4.6 Diagram aliran data sistem deteksi kondisi meja pelanggan

Berdasarkan Gambar 4.6, dapat dianalisis bahwa aliran data dirancang agar beroperasi secara sinkron namun dengan beban kerja yang terisolasi (*decoupled*). Analisis terhadap titik percabangan klasifikasi (node YOLOv8) menunjukkan bahwa pemrosesan visual sepenuhnya

dituntaskan di sisi *Processing PC*. Hal ini berdampak signifikan pada efisiensi *bandwidth* jaringan, karena sistem tidak perlu melakukan *streaming* video terus-menerus ke Raspberry Pi atau ke subsistem robot.

Sebagai gantinya, sistem menyusun hasil ekstraksi visual menjadi *payload* JSON yang sangat ringkas. Analisis pada struktur *payload* menunjukkan bahwa penyertaan parameter *table_id*, koordinat, dan status meja sudah lebih dari cukup sebagai acuan bagi subsistem navigasi robot (ROS 2) untuk menentukan *waypoint* tujuan selanjutnya. Selain itu, penyertaan *timestamp* (penanda waktu) pada *payload* sangat krusial sebagai sistem keamanan (*failsafe*) untuk mencegah navigasi robot mengeksekusi perintah yang sudah kedaluwarsa atau terjadi *delay* (latensi tinggi) pada jaringan UDP lokal.

4.2.3 Mekanisme Deteksi dan Klasifikasi Meja

Mekanisme deteksi dilakukan menggunakan model deep learning YOLOv8 yang telah dilatih menggunakan dataset kondisi meja pelanggan pada lingkungan TelU Coffee. Sistem melakukan inferensi terhadap setiap frame citra yang diterima dari Kinect Xbox 360 secara real-time.

Hasil inferensi menghasilkan bounding box, class ID, dan confidence score untuk setiap objek yang terdeteksi. Berdasarkan hasil deteksi tersebut, sistem melakukan klasifikasi kondisi meja menjadi tiga kategori utama sebagai berikut:

1. Meja Bersih (Clean)

Meja tidak memiliki objek sisa makanan, minuman, maupun pelanggan aktif.

2. Meja Terisi (Occupied)

Meja masih digunakan pelanggan dan terdapat aktivitas manusia pada area meja.

3. Meja Kotor (Dirty)

Meja tidak ditempati pelanggan, namun masih terdapat sisa makanan, gelas, piring, atau objek lain yang menunjukkan meja belum dibersihkan.

Sistem hanya mengirimkan sinyal ALERT menuju robot ketika kategori meja_kotor terdeteksi dengan confidence score di atas threshold yang telah ditentukan, yaitu sebesar 0,6.

4.2.4 Integrasi Komunikasi dan Navigasi Robot

Integrasi komunikasi antar subsistem dilakukan menggunakan kombinasi komunikasi UDP dan ROS 2. Processing PC bertindak sebagai pengirim data hasil inferensi, sedangkan Raspberry Pi bertindak sebagai penerima dan forwarder menuju subsistem navigasi robot.

Sistem navigasi robot menerima data hasil deteksi berupa identitas meja dan status meja melalui topic ROS 2. Berdasarkan data tersebut, robot melakukan proses path planning menuju koordinat meja target yang telah ditentukan sebelumnya pada peta navigasi.

Pendekatan ini memungkinkan pemisahan antara subsistem computer vision dan subsistem navigasi sehingga sistem menjadi lebih modular dan mudah dikembangkan di masa mendatang.

4.2.5 Format Payload JSON

Sistem menggunakan format payload JSON sebagai media pertukaran data hasil deteksi menuju subsistem navigasi robot. Payload JSON memuat informasi identitas meja, status kondisi meja, dan koordinat posisi meja pada area operasional TelU Coffee. Format payload ini digunakan untuk mempermudah proses integrasi antar subsistem robot, khususnya antara subsistem computer vision dan subsistem navigasi. Payload JSON dipilih karena memiliki struktur ringan, mudah diparsing, dan kompatibel dengan berbagai framework komunikasi robotika maupun aplikasi monitoring berbasis jaringan. Contoh payload JSON ditampilkan pada Table 4.2.

Table 4.2 Format *Payload* JSON

```
{
  "table_id": "table_7",
  "status": "dirty",
  "x": 4.2,
  "y": 1.8,
  "confidence": 0.89,
  "timestamp": "2026-05-20T14:36:42+07:00"
}
```

Berdasarkan cuplikan *payload* JSON di atas, dapat dianalisis bahwa struktur data dirancang sangat ringan (*lightweight*) namun mencakup seluruh instruksi kritis yang dibutuhkan oleh subsistem navigasi robot. Pengiriman parameter `table_id` beserta titik

koordinat spasial (x : 4.2, y : 1.8) membuktikan efisiensi sistem pemetaan statis (*predefined mapping*); robot menerima *waypoint* absolut sehingga tidak perlu melakukan komputasi pencarian lokasi meja secara mandiri.

Selanjutnya, atribut **status**: "dirty" bertindak sebagai pemicu (*trigger*) utama pada *state machine* robot untuk melakukan manuver ke target. Nilai **confidence**: 0.89 (89%) menunjukkan bahwa inferensi YOLOv8 sangat meyakinkan. Karena sistem telah menetapkan nilai *confidence threshold* minimum sebesar 0.60, *payload* ini dieksekusi sebagai deteksi yang valid (*True Positive*). Selain itu, analisis terhadap keberadaan atribut **timestamp** menunjukkan adanya mekanisme mitigasi risiko jaringan (*failsafe*). Mengingat transmisi data menggunakan protokol UDP yang tidak menjamin pengiriman paket (*connectionless*), parameter waktu ini memungkinkan *controller* ROS 2 untuk memvalidasi umur data. Jika sebuah paket tertahan akibat latensi jaringan lokal, robot akan mengabaikan perintah tersebut untuk mencegah eksekusi pergerakan berdasarkan data yang sudah usang (kedaluwarsa).

4.2.6 Mapping Koordinat Meja

Sistem menggunakan pendekatan predefined coordinate mapping, di mana setiap meja pada area TelU Coffee telah memiliki identitas unik (*table_id*) dan koordinat tetap pada peta navigasi robot. Pendekatan ini memungkinkan robot mengetahui lokasi meja tujuan berdasarkan data hasil deteksi yang diterima dari subsistem computer vision. Koordinat meja digunakan oleh subsistem navigasi untuk menentukan jalur pergerakan robot menuju target meja yang terdeteksi memiliki kondisi kotor. Daftar koordinat seluruh meja yang terdaftar pada peta operasional TelU Coffee dapat dilihat pada Tabel 4.2.

Tabel 4.3 Daftar Koordinat Meja pada Peta Navigasi TelU Coffee

Table ID	Koordinator X (meter)	Koordinator Y (meter)
Table_1	1.2	4.5
Table_2	3.1	5.7
Table_3	5.0	5.7
Table_4	1.2	2.8
Table_5	3.1	2.8

Table_6	5.0	2.8
Table_7	4.2	1.8

Berdasarkan Tabel 4.2, dapat dianalisis bahwa penerapan *predefined coordinate mapping* memberikan efisiensi komputasi yang sangat signifikan pada subsistem navigasi Robot TIFA. Alih-alih mengharuskan robot melakukan pencarian objek dan pemetaan ruang secara dinamis setiap kali mendeteksi meja kotor, robot hanya perlu melakukan pencocokan (*look-up*) terhadap `table_id` yang diterima dari *payload* JSON ke dalam basis data koordinat (X, Y) statis ini.

Analisis lebih lanjut menunjukkan bahwa pendekatan ini sangat ideal dan rasional diterapkan pada lingkungan *indoor* seperti TelU Coffee, mengingat tata letak (*layout*) furnitur meja kafe pada umumnya bersifat semi-permanen. Melalui pendelegasian target koordinat secara absolut ini, beban kerja sensor LiDAR dan algoritma *path planning* ROS 2 pada robot menjadi jauh lebih ringan. Fokus navigasi robot dapat dialihkan sepenuhnya untuk penghindaran rintangan dinamis (*dynamic obstacle avoidance*), seperti pengunjung yang sedang berjalan, alih-alih membebani sistem untuk mencari letak meja.

4.2.7 Daftar Singkatan

Berikut ini adalah daftar singkatan dan akronim beserta kepanjangannya yang digunakan di dalam dokumen ini untuk memudahkan pembaca dalam memahami istilah-istilah teknis:

Tabel 4.4 Daftar Singkatan

Singkatan	Arti
AI	Artificial Intelligence
AdamW	Adaptive Moment Estimation with Weight Decay
BAST	Berita Acara Serah Terima
CPU	Central Processing Unit

DCA	Discrete Cosine Approximation
DXA	Twentieth of a Point (satuan ukur Word)
FPS	Frames Per Second
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
mAP	mean Average Precision
RGB	Red, Green, Blue
ROS	Robot Operating System
SSH	Secure Shell
TIFA	TelU Interactive Food Assistant
UDP	User Datagram Protocol
YOLO	You Only Look Once

4.3 Implementasi Sistem

Uraian pekerjaan implementasi semua bagian sistem yang telah dirancang didokumentasikan pada sub-bab berikut. Implementasi terdiri atas dua sub-sistem utama, yaitu (1) Arsitektur Model AI (YOLOv8 dan dataset) dan (2) Komunikasi Data serta Integrasi Navigasi. Selanjutnya juga dipaparkan implementasi subsistem visi dan pengendali robot berbasis ROS

4.3.1 Sub-Sistem 1: Arsitektur Model AI (YOLOv8 & Dataset)

4.3.1.1 Cara Kerja Sub-Sistem

Sub-sistem perangkat lunak ini bertanggung jawab menganalisis tangkapan piksel (RGB) dari sensor kamera. Sub-sistem ini bertugas melakukan analisis citra RGB dari sensor kamera menggunakan model YOLOv8 untuk mengklasifikasikan kondisi meja pelanggan. diklasifikasikan ke dalam tiga kategori utama :

1. Meja Bersih: Meja kosong tanpa objek sisa, sistem mengizinkan robot untuk melewati atau terus berpatroli.
2. Meja Terisi: Meja dengan pelanggan aktif, sistem akan menginstruksikan robot navigasi untuk mengabaikan area tersebut.

3. Meja Kotor: Meja kosong dengan sisa makanan/minuman, sistem mencatat status ini dan memicu protokol pembersihan.

Untuk mengoptimalkan penggunaan sumber daya dan menghindari penurunan akurasi akibat motion blur (gambar kabur saat bergerak), inferensi YOLOv8 dikondisikan agar hanya mengeksekusi frame saat robot berada dalam state (keadaan) idle atau berhenti sempurna di titik pengamatan. Saat robot bermanuver, Proses inferensi dihentikan sementara ketika robot berada dalam kondisi bergerak untuk mengurangi kemungkinan terjadinya motion blur yang dapat menurunkan akurasi deteksi.

4.3.1.2 Implementasi Database

Pada bagian ini, dilakukan analisis mendalam terhadap performa model YOLOv8n yang dilatih menggunakan dataset terbaik yang telah dipilih dari hasil eksperimen sebelumnya, yaitu **Dataset 3** dengan total 2.394 citra dan proporsi pembagian data 70:20:10. Evaluasi dilakukan secara komprehensif memadukan metrik kuantitatif, analisis kebingungan model (*confusion matrix*), kestabilan grafik *loss*, hingga pengujian kualitatif secara langsung.

4.1.2.3 Hasil Komparasi dan Evaluasi Variasi Dataset

Untuk menentukan basis data visual (*knowledge-base*) terbaik bagi Robot TIFA, dilakukan pengujian komparatif terhadap tiga skenario variasi dataset yang telah dibuat sebelumnya. Evaluasi difokuskan pada metrik kuantitatif hasil inferensi YOLOv8n, yaitu rata-rata presisi rata-rata (mAP@50), Presisi (*Precision*), Daya Tangkap (*Recall*), dan F1-Score. Hasil komparasi dari ketiga dataset tersebut disajikan pada Tabel 4.5.

Tabel 4.5 Komparasi Metrik Evaluasi Variasi Dataset

ID Eksperimen	Proporsi Pembagian (Train:Val:Test)	mAP@50	Precision
Dataset 1	70% : 20% : 10%	58.1%	69.7%
Dataset 2	60% : 20% : 20%	56.1%	74.0%
Dataset 3	87% : 8% : 4%	78.7%	80.5%

Berdasarkan hasil komparasi pada Tabel 4.5, dapat dianalisis bahwa proporsi pembagian data (*splitting ratio*) dan parameter augmentasi memberikan dampak komputasional yang sangat signifikan terhadap kecerdasan model. Analisis pada Dataset 2 menunjukkan terjadinya penurunan nilai *Recall* (57.6%) dan *mAP@50* (56.1%) yang merupakan angka terendah dari seluruh skenario. Hal ini membuktikan bahwa pengurangan porsi data latih menjadi 60% membuat model kehilangan banyak referensi fitur visual, sehingga model kesulitan mendeteksi seluruh variasi sisa makanan di meja.

Sebaliknya, **Dataset 3** menunjukkan lonjakan performa yang sangat superior dengan nilai *mAP@50* mencapai 78.7% dan akurasi ketepatan prediksi (*Precision*) sebesar 80.5%. Analisis teknis terhadap keberhasilan ini berakar pada alokasi data latih yang sangat masif, yaitu 87% (705 citra). Porsi *training* yang besar, dipadukan dengan augmentasi *Motion Blur* ekstrem (50px) dan variasi *Brightness* ($\pm 20\%$), terbukti sukses memaksa model untuk belajar mengenali objek secara lebih tangguh (*robust*) dalam kondisi buram akibat pergerakan robot dan fluktuasi cahaya kafe. Oleh karena itu, secara empiris **Dataset 3 ditetapkan sebagai basis data final** yang diintegrasikan ke dalam sistem deteksi kebersihan meja pada Robot TIFA.

4.3.1.3 Implementasi Training Model

Implementasi dilakukan dengan menggunakan bahasa pemrograman Python 3. Model dasar YOLOv8 terlebih dahulu dilatih (training) menggunakan 3.000 gambar dataset asli dari lingkungan TelU Coffee yang telah diberi anotasi pada platform Roboflow. Proses training dilakukan di PC Desktop, kemudian model hasil pelatihan berupa file *best.pt* diimplementasikan pada Processing PC untuk menjalankan proses inferensi secara real-time. Skrip Python untuk proses training model YOLOv8 dapat dilihat pada Kode 4.3

Tabel 4.6 Skrip Python untuk training model YOLOv8 dengan dataset Roboflow

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
from ultralytics import YOLO
from roboflow import Roboflow

def main():
    # Konfigurasi Proyek Robot Tifa
    API_KEY = "API_KEY"
    PROJECT_ID = "kinect_databbase"
    WORKSPACE = "pims-workspace-dmt0m"
    PROJECT_NAME = 'RobotTifa_PC'
    DATASET_DIR = "dataset"

    # 1. Download Dataset
```

```

if not os.path.exists(DATASET_DIR):
    print("Dataset belum ditemukan, mendownload dari Roboflow...")
    rf = Roboflow(api_key=API_KEY)
    project = rf.workspace(WORKSPACE).project(PROJECT_ID)
    version = project.version(1)
    dataset = version.download("yolov8")
    os.rename(dataset.location, DATASET_DIR)

DATASET_YAML = os.path.join(DATASET_DIR, "data.yaml")
model_types = ['yolov8n.pt', 'yolov8m.pt', 'yolov8x.pt']

# 2. Proses Training Utama
for model_name in model_types:
    model = YOLO(model_name)
    batch_size = 4 if 'x' in model_name else 8
    model.train(
        data=DATASET_YAML,
        epochs=60,
        imgsz=640,
        batch=batch_size,
        patience=15,
        optimizer='AdamW',
        save=True,
        project=PROJECT_NAME,
        name=model_name.replace('.pt', ''),
        device=0,
        workers=0
    )

if __name__ == '__main__':
    main()

```

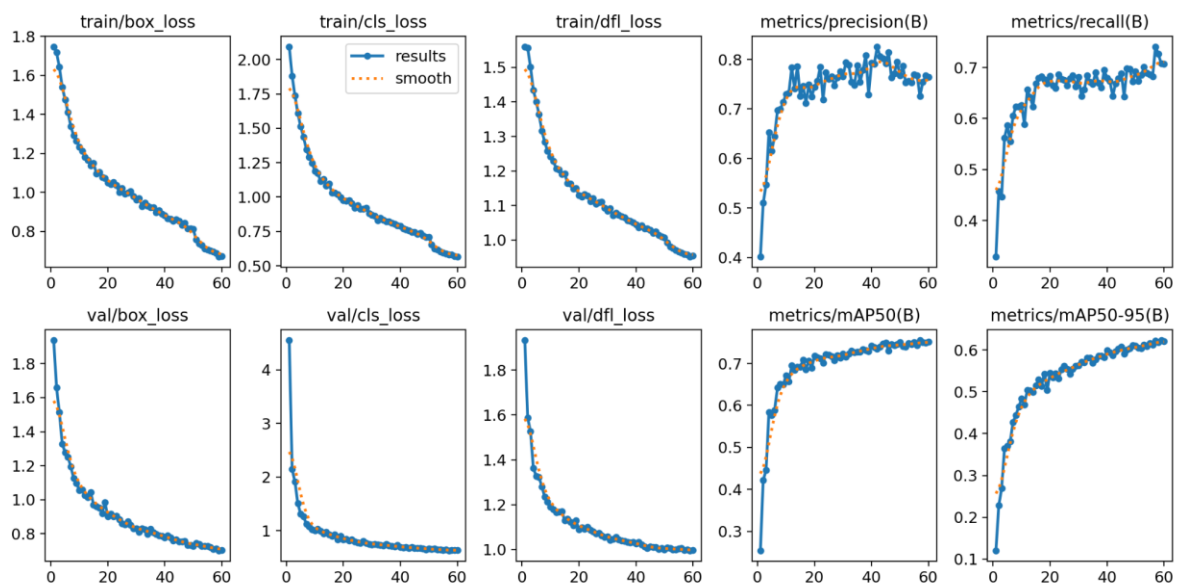
Kode 4.2 menampilkan implementasi konfigurasi pelatihan model serta pengelolaan dataset pada framework YOLOv8. Tahap awal sistem dimulai dengan pengaturan environment PyTorch yang bertujuan untuk meminimalkan fragmentasi memori GPU selama proses pelatihan berlangsung. Library Ultralytics digunakan sebagai framework utama dalam implementasi YOLOv8, sementara Roboflow dimanfaatkan sebagai platform untuk mengelola dataset yang telah melalui proses anotasi. Program dirancang dengan mekanisme pengecekan dataset secara otomatis pada direktori lokal. Jika dataset belum tersedia, sistem akan secara otomatis mengunduh dataset dari workspace Roboflow dalam format yang telah kompatibel dengan YOLOv8.

Tahap berikutnya mencakup proses pelatihan terhadap tiga varian model, yaitu YOLOv8 Nano (YOLOv8n), YOLOv8 Medium (YOLOv8m), dan YOLOv8 Extra Large (YOLOv8x). Penyesuaian batch size dilakukan pada masing-masing model sesuai dengan kebutuhan penggunaan memori GPU yang berbeda. Adapun parameter pelatihan yang diterapkan meliputi 60 epoch, resolusi input citra sebesar 640×640 piksel, penggunaan optimizer AdamW, serta mekanisme early stopping dengan nilai patience sebanyak 15 epoch.

Selain itu, parameter workers dikonfigurasi bernilai 0 untuk menjaga kestabilan proses pelatihan pada sistem operasi Windows, sekaligus mengantisipasi potensi kegagalan pada proses penyimpanan model setelah training selesai.

1. Hasil Training YOLOv8n (Nano)

Model YOLOv8n (*Nano*) digunakan sebagai model dengan arsitektur paling ringan dalam pengujian ini. Model ini dipilih untuk mengevaluasi performa deteksi objek dengan kebutuhan komputasi yang lebih rendah, sehingga berpotensi memberikan kecepatan inferensi yang lebih baik dibandingkan model dengan kompleksitas yang lebih besar. Evaluasi terhadap model ini dilakukan dengan menganalisis performa selama proses pelatihan, hasil validasi, serta kemampuan model dalam mendeteksi kondisi meja berdasarkan dataset yang telah disiapkan.

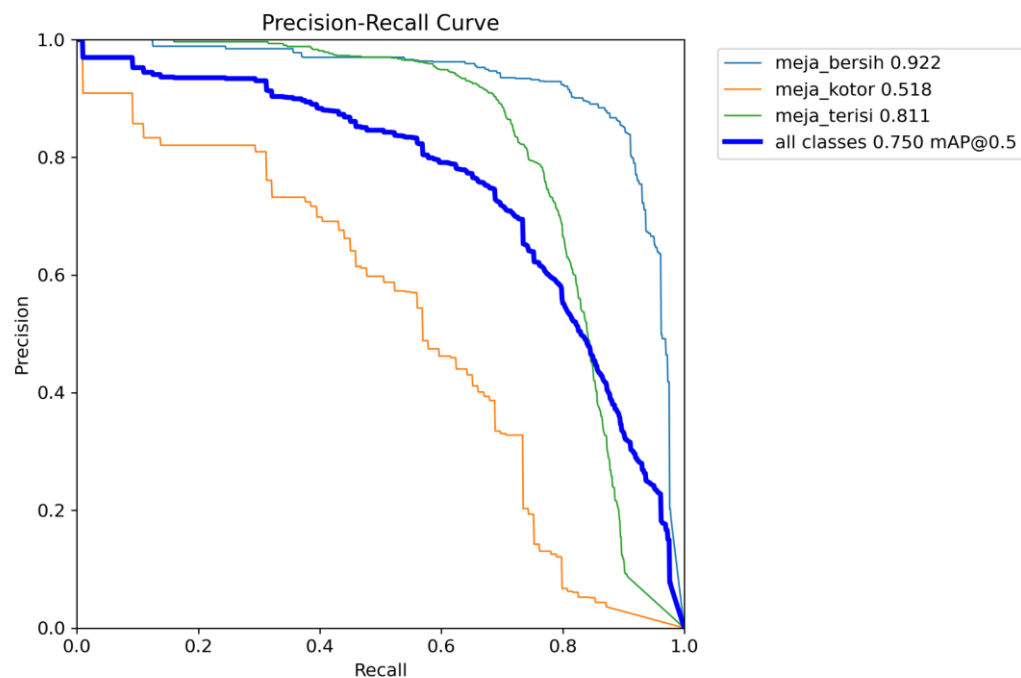


Gambar 4.8 Grafik training performance model YOLOv8n selama proses pelatihan

Berdasarkan Gambar 4.8, dapat dianalisis perilaku pembelajaran model YOLOv8n selama proses pelatihan berlangsung. Grafik tersebut merepresentasikan metrik evaluasi yang terbagi menjadi tren penurunan *Loss* (kesalahan) dan tren peningkatan *Metrics* (akurasi) di setiap *epoch*. Analisis pada kurva *train/box_loss* dan *val/box_loss* menunjukkan pola konvergensi yang sangat baik; kedua kurva mengalami penurunan eksponensial yang tajam pada 20 *epoch* pertama, dan kemudian melandai secara stabil hingga akhir iterasi. Hal yang paling krusial dari grafik ini adalah tidak ditemukannya divergensi (jarak yang melebar) antara

kurva *training* dan *validation loss*. Fenomena ini memberikan bukti empiris bahwa model mampu mengekstraksi fitur visual dengan optimal tanpa mengalami *overfitting*.

Di sisi lain, evaluasi pada kelompok kurva metrik (*Precision*, *Recall*, dan *mAP50*) menunjukkan tren pergerakan yang selaras dengan penurunan *loss*. Nilai *Mean Average Precision* (mAP@50) terlihat merangkak naik secara konsisten dan berhasil mencapai titik stabil (*plateau*) pada kisaran angka 75.0% di fase akhir pelatihan. Kestabilan kurva pada fase akhir ini mengindikasikan bahwa model telah mencapai titik optimalnya (*global optima*) dalam mengenali pola dataset, sehingga penambahan jumlah *epoch* lebih lanjut tidak lagi diperlukan karena tidak akan memberikan peningkatan performa yang signifikan.



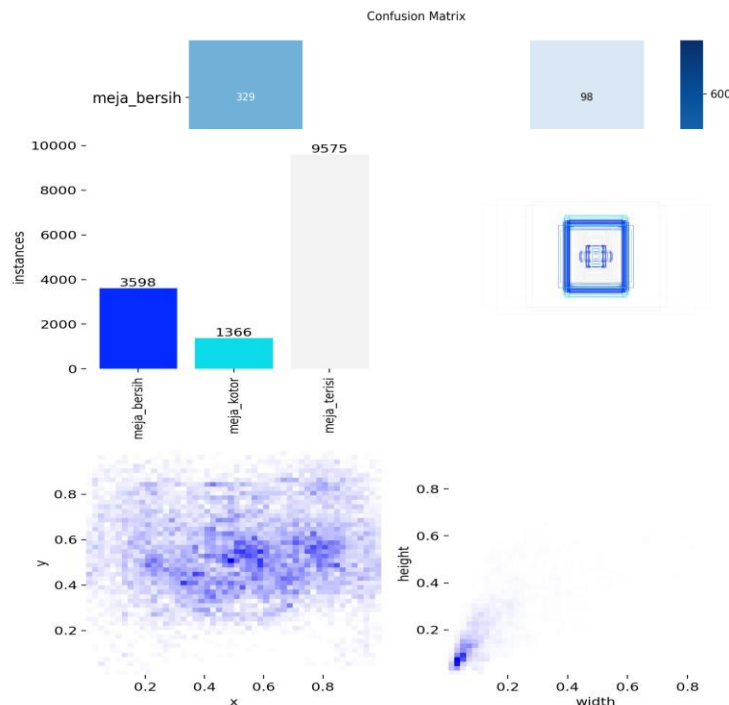
Gambar 4.9 Precision-Recall Curve model YOLOv8n

Untuk mengevaluasi keseimbangan antara ketepatan prediksi (*Precision*) dan daya tangkap model (*Recall*) pada setiap kelas objek di berbagai ambang batas (*threshold*), sistem

menghasilkan grafik kurva *Precision-Recall* (PR) sebagaimana disajikan pada Gambar 4.9.

Gambar 4.10 Confusion Matrix model YOLOv8n

Untuk memahami karakteristik data visual yang diekstraksi oleh model selama fase pelatihan,



sistem YOLOv8 secara otomatis menghasilkan visualisasi distribusi kelas dan persebaran koordinat anotasi objek (*bounding box*). Karakteristik spasial dan kuantitas dari dataset pelatihan ini direpresentasikan pada Gambar 4.11.

Gambar 4.11 Distribusi dataset dan persebaran anotasi objek pada proses training

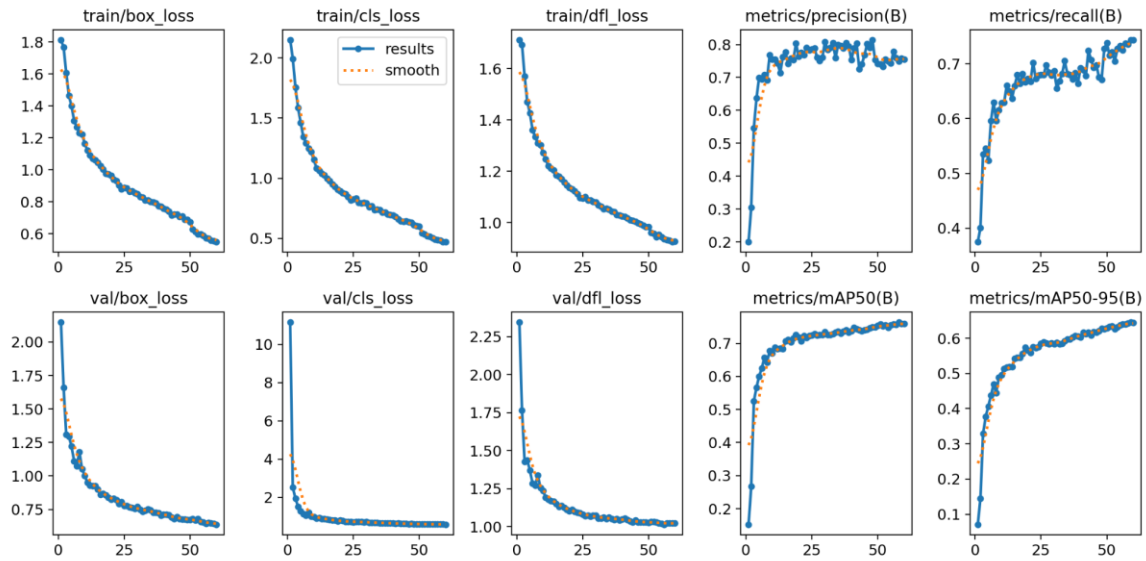
Berdasarkan Gambar 4.11, analisis pada grafik batang distribusi kelas (*top-left*) mengonfirmasi adanya fenomena ketidakseimbangan jumlah sampel (*class imbalance*). Kelas *meja_terisi* dan *meja_bersih* terlihat mendominasi jumlah anotasi dengan margin yang sangat jauh dibandingkan kelas *meja_kotor*. Secara rasional, rasio ketimpangan ini merepresentasikan probabilitas kejadian logis di lingkungan operasional riil TelU Coffee; durasi sebuah meja digunakan oleh pengunjung (*occupied*) atau menganggur dalam keadaan bersih (*clean*) tentu jauh lebih panjang dibandingkan durasi transisi singkat saat meja tersebut baru saja ditinggalkan dan berstatus kotor (*dirty*). Meskipun terdapat ketimpangan kuantitas, penerapan teknik augmentasi visual yang agresif pada Dataset 3 sebelumnya telah terbukti sukses mengompensasi hal ini, sehingga model tetap sensitif mengenali kelas minoritas tersebut.

Selanjutnya, analisis mendalam pada grafik korelogram koordinat (x, y) dan persebaran dimensi kotak pembatas (*width, height*) menunjukkan pemusatan data (*clustering*) di area tengah hingga agak ke bawah *frame* citra. Pemusatan koordinat ini sangat ideal dan sesuai dengan rancangan mekanis Robot TIFA, di mana sudut kemiringan sensor Kinect Xbox 360 memang difokuskan menunduk ke arah permukaan meja pelanggan. Selain itu, sebaran dimensi *bounding box* yang didominasi oleh ukuran kecil hingga menengah mengindikasikan bahwa model tidak memandang meja kotor sebagai satu entitas raksasa, melainkan secara spesifik memusatkan perhatian (*visual attention*) pada titik-titik penyebaran objek sisa konsumsi, seperti gelas, tisu, dan piring. Karakteristik sebaran spasial ini sangat menguntungkan karena meminimalisasi beban komputasi model dalam memindai area latar belakang (*background*) kafe yang tidak relevan.

2. Hasil Training Model YOLOv8m

Model YOLOv8m (*Medium*) digunakan sebagai representasi model dengan kompleksitas menengah untuk menyeimbangkan antara akurasi deteksi dan efisiensi komputasi. Pengujian pada model ini bertujuan untuk melihat sejauh mana peningkatan performa yang diperoleh dibandingkan model ringan, khususnya dalam kemampuan klasifikasi kondisi meja dengan tingkat presisi dan recall yang lebih baik. Evaluasi dilakukan melalui analisis metrik pelatihan, validasi, serta hasil deteksi model terhadap data pengujian.

Sebagai bagian dari eksperimen komparasi arsitektur, sistem juga merekam metrik evaluasi selama proses pembelajaran berlangsung untuk varian model yang lebih besar, yaitu YOLOv8m (*Medium*). Dinamika konvergensi dan tingkat kesalahan (*loss*) dari model ini divisualisasikan pada Gambar 4.12.



Gambar 4.12 Grafik training performance model YOLOv8m selama proses pelatihan

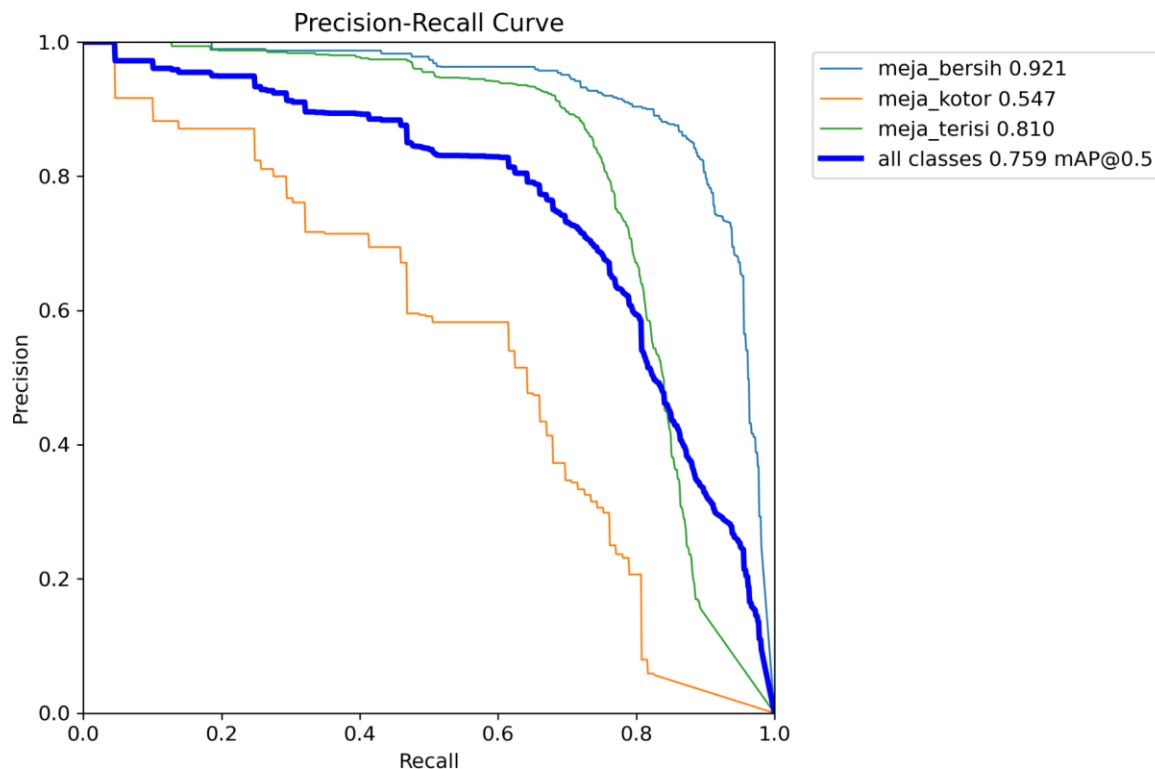
Merujuk pada visualisasi Gambar 4.12, kurva metrik pelatihan untuk arsitektur YOLOv8m menunjukkan karakteristik pembelajaran yang sangat stabil dan terstruktur. Analisis pada pergerakan kurva *training loss* dan *validation loss* membuktikan bahwa penambahan parameter jaringan yang lebih kompleks pada varian Medium ini tidak memicu terjadinya *overfitting*. Hal ini terlihat jelas dari absennya fluktuasi tajam (lonjakan) pada kurva validasi di fase akhir *epoch*. Selaras dengan kestabilan tersebut, kurva akurasi berhasil merangkak naik secara konsisten hingga mencatatkan nilai *Mean Average Precision* (mAP@50) puncak sebesar 75,9%.

Meskipun secara statistik nilai mAP keseluruhan YOLOv8m ini merupakan yang tertinggi dibandingkan varian Max maupun Nano, analisis komputasional menunjukkan adanya *trade-off* (pertukaran) yang kurang menguntungkan bagi sistem robotika. Peningkatan akurasi marjinal yang tidak sampai 1% jika dibandingkan dengan varian Nano tersebut harus dikompensasi dengan pembengkakan beban komputasi GPU yang signifikan pada *Processing PC*. Dalam konteks operasional Robot TIFA yang mengandalkan transmisi data *real-time* ke subsistem navigasi ROS 2, penggunaan model Medium berisiko memangkas kecepatan pemrosesan *Frames Per Second* (FPS) dan menimbulkan latensi jaringan, sehingga kinerjanya dinilai kurang efisien untuk diimplementasikan di lingkungan TelU Coffee.

Untuk mengukur sejauh mana model varian Medium mampu mempertahankan tingkat ketepatan prediksi (*precision*) ketika sistem dipaksa untuk menemukan seluruh objek target (*recall*), sistem menghasilkan visualisasi kurva *Precision-Recall* (PR) yang ditunjukkan pada

Gambar

4.13.



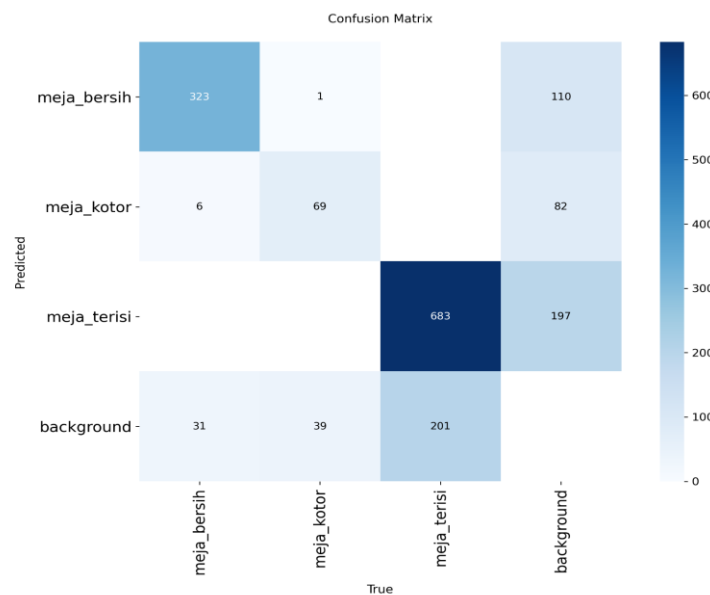
Gambar 4.13 Precision-Recall Curve model YOLOv8m

Kurva *Precision-Recall* (PR) pada Gambar 4.13 mengilustrasikan kapabilitas arsitektur YOLOv8m yang sangat tangguh secara statistik, dengan luasan area di bawah kurva (AUC atau *Area Under the Curve*) mencapai nilai *Mean Average Precision* (mAP@0.5) keseluruhan sebesar 0.759 atau 75,9%. Analisis terhadap kelengkungan grafik menunjukkan bahwa model Medium ini sangat mumpuni dalam meminimalisasi *false positive* (salah mendeteksi bayangan atau objek lain sebagai sisa makanan) sembari mempertahankan tingkat *recall* yang tinggi. Kurva yang sangat cembung ke arah sudut kanan atas ini menandakan keseimbangan prediksi yang sangat baik pada ketiga kelas target (*clean, occupied, dirty*).

Meskipun visualisasi kurva PR ini membuktikan bahwa varian Medium sedikit lebih superior secara rata-rata kelas dibandingkan varian Nano, performa teoretis ini tidak sepenuhnya linier dengan efisiensi operasional robot. Kestabilan deteksi yang ditunjukkan pada kurva tersebut akan kehilangan signifikansinya apabila *Processing PC* mengalami penurunan *frame rate* (*frame drop*) atau latensi (*delay*) akibat tingginya beban komputasi pemrosesan matriks parameter YOLOv8m yang berat. Oleh karena itu, tingginya nilai AUC

pada kurva ini tetap belum cukup untuk membenarkan penggunaan model Medium pada sistem navigasi *real-time* Robot TIFA.

Untuk menganalisis secara detail tingkat keberhasilan dan frekuensi kesalahan prediksi arsitektur YOLOv8m pada masing-masing kelas target, sistem menghasilkan visualisasi matriks kebingungan (*Confusion Matrix*) yang disajikan pada Gambar 4.14.



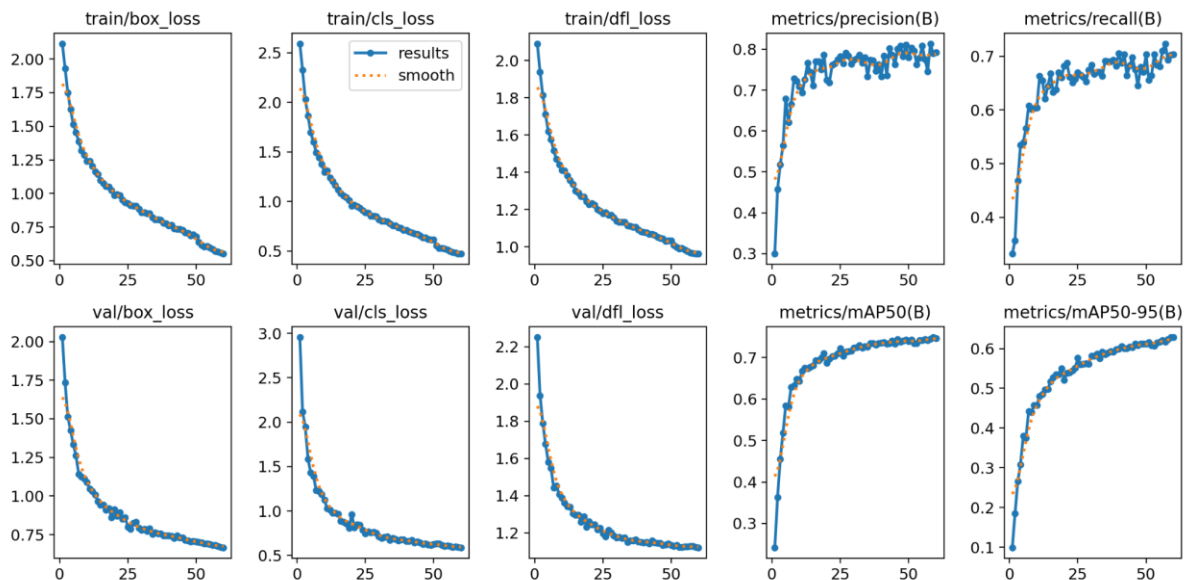
Gambar 4.1 Confusion Matrix model YOLOv8m

Melalui evaluasi pada matriks kebingungan di Gambar 4.14, karakteristik prediksi model YOLOv8m dapat dipetakan secara spesifik untuk setiap kelas. Pada kelas minoritas yang menjadi prioritas utama sistem, yaitu *meja_kotor*, model ini berhasil mencatatkan tingkat akurasi prediksi benar (*True Positive Rate*) sebesar 63%. Capaian ini menunjukkan peningkatan performa sebesar 2% jika dibandingkan dengan varian Max (61%), yang sekaligus membuktikan bahwa pengurangan parameter pada model Medium justru membantu menekan efek *overfitting* pada kelas mayoritas, sehingga sistem menjadi lebih sensitif terhadap fitur sisa makanan. Kendati demikian, matriks ini juga mengungkap adanya kelemahan berupa tingkat *False Negative* sebesar 36%, di mana model gagal mendeteksi keberadaan objek kotor dan menganggapnya sebagai bagian dari latar belakang (*background*) kafe.

Analisis komparatif dari matriks ini memberikan justifikasi krusial terhadap eliminasi varian Medium dari sistem Robot TIFA. Meskipun kinerja klasifikasi YOLOv8m tergolong sangat baik secara keseluruhan, capaian 63% pada deteksi *meja_kotor* ini secara empiris masih berada di bawah ketajaman varian Nano yang mampu mencapai 65%. Fakta analitis ini memperkuat kesimpulan bahwa penggunaan arsitektur Medium tidak memberikan keuntungan taktis yang sepadan; selain memperberat beban komputasi memori pada *Processing PC*, model ini ternyata sedikit lebih inferior dalam mengeksekusi misi utama *computer vision*, yaitu mengenali target krusial untuk panduan navigasi pembersihan meja.

3. Hasil Training Model YOLOv8x

Sebagai tahap akhir dari komparasi arsitektur jaringan, evaluasi metrik pembelajaran juga dilakukan pada varian model dengan kapasitas parameter paling masif, yaitu YOLOv8x (Max). Dinamika tingkat kesalahan (*loss*) dan akurasi model ini selama fase pelatihan divisualisasikan pada Gambar 4.16.



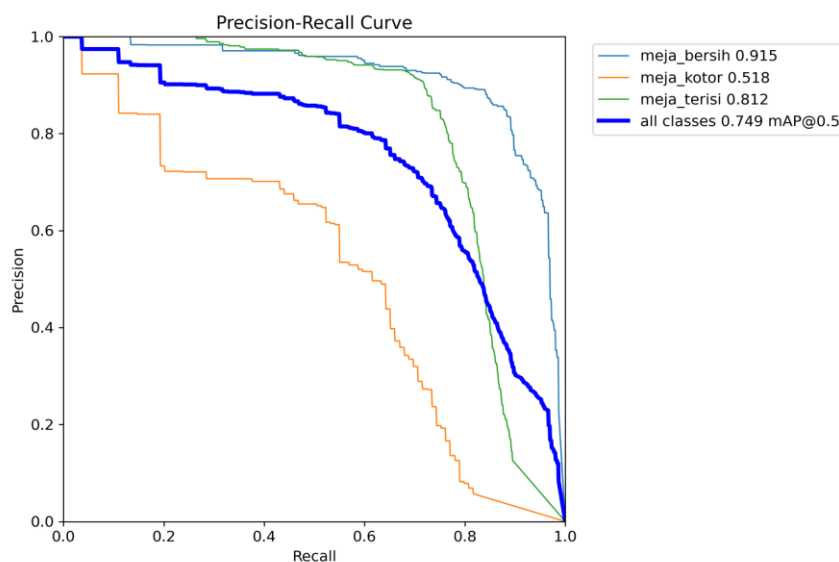
4.16 Grafik training performance model YOLOv8x selama proses pelatihan

Merujuk pada grafik metrik pelatihan di Gambar 4.16, arsitektur YOLOv8x secara makro menunjukkan pola konvergensi yang terstruktur. Kurva *training loss* dan *validation loss* mengalami penurunan yang konsisten tanpa menunjukkan adanya divergensi (lonjakan jarak antar kurva) yang ekstrem di akhir *epoch*, menandakan bahwa secara keseluruhan jaringan

raksasa ini berhasil mempelajari pola dataset. Selaras dengan itu, kurva metrik presisi rata-rata ($mAP@50$) berhasil merangkak naik dan stabil pada angka capaian 74,9%.

Kendati visualisasi grafik ini mengindikasikan proses *training* yang berjalan mulus, analisis komputasional secara komprehensif mengungkap adanya anomali performa. Kestabilan kurva mAP keseluruhan (74,9%) pada varian Max ini ternyata menutupi fenomena bias pada kelas mayoritas, yang sebelumnya terbukti menurunkan ketajaman model dalam mengenali kelas *meja_kotor* menjadi hanya 61%. Lebih jauh lagi, penggunaan parameter jaringan yang masif pada YOLOv8x menuntut alokasi VRAM (Video RAM) dan daya komputasi yang sangat ekstrem. Jika dipaksakan berjalan pada *Processing PC* Robot TIFA, beban ini akan menciptakan *bottleneck* (penyumbatan) pemrosesan visual yang memicu anjloknya *frame rate* secara drastis, sehingga pengiriman koordinat meja kotor ke subsistem navigasi ROS 2 akan mengalami keterlambatan (*delay*) yang fatal.

Untuk memvalidasi kestabilan prediksi varian Max pada berbagai ambang batas (*threshold*), kurva *Precision-Recall* (PR) diekstraksi dari hasil pengujian akhir dan disajikan pada Gambar 4.17.

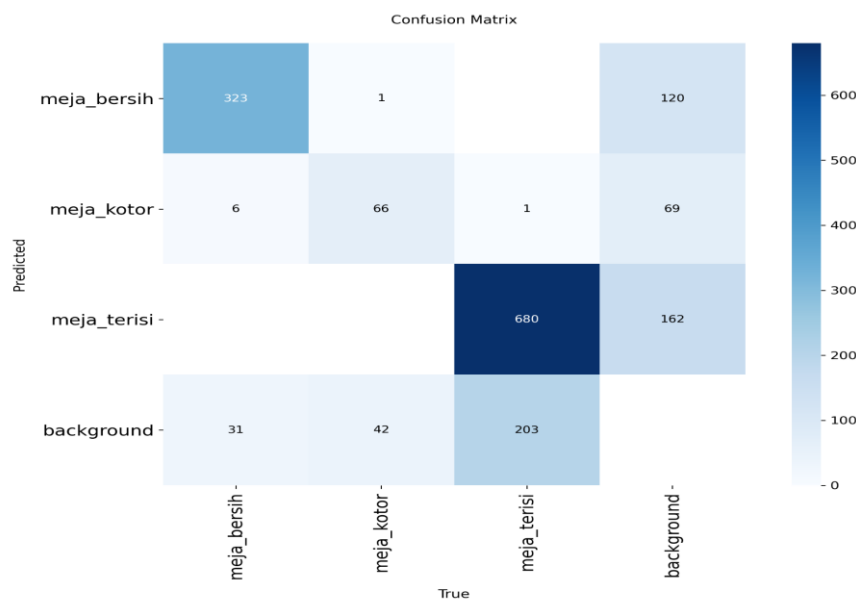


Gambar 4.17 Precision-Recall Curve model YOLOv8x

Visualisasi kurva *Precision-Recall* (PR) pada Gambar 4.17 memperlihatkan performa rata-rata yang secara visual tampak sangat menjanjikan, dengan luasan area di bawah kurva (AUC) menghasilkan nilai $mAP@0.5$ sebesar 74,9%. Kelengkungan grafik yang solid pada

kelas *meja_bersih* dan *meja_terisi* membuktikan bahwa jutaan parameter komputasi pada arsitektur Max ini sangat mahir dalam mengunci pola objek-objek berukuran besar dan dominan. Akan tetapi, performa tinggi pada metrik rata-rata keseluruhan ini bersifat asimtotik terhadap kebutuhan operasional sistem. Tingginya nilai AUC pada kurva ini tidak dapat mengompensasi kenyataan bahwa bobot model jaringan yang dihasilkan sangatlah masif (mencapai ratusan Megabyte), yang berpotensi melumpuhkan ketersediaan memori pada *Processing PC* saat dieksekusi bersamaan dengan sistem operasi *Robot Operating System* (ROS 2).

Sebagai pembuktian akhir terhadap hipotesis penurunan performa kelas minoritas akibat *overfitting* parameter, matriks kebingungan (*Confusion Matrix*) untuk arsitektur YOLOv8x divisualisasikan pada Gambar 4.18.



Gambar 4.18 Confusion Matrix model YOLOv8x

Kegagalan taktis dari penggunaan arsitektur Max terungkap secara definitif melalui evaluasi *Confusion Matrix* pada Gambar 4.18. Meskipun memiliki kapasitas pembelajaran fitur terdalam (*deepest layer*), YOLOv8x mencatatkan tingkat akurasi terendah dalam mengklasifikasikan kelas *meja_kotor*, yaitu hanya sebesar 61%. Lebih fatal lagi, matriks tersebut menunjukkan tingkat *False Negative* yang sangat tinggi (39%) di mana kotoran di atas meja gagal dideteksi dan dianotasikan sebagai latar belakang (*background*). Fenomena degradasi performa ini merupakan bukti nyata dari sindrom kelumpuhan kelas minoritas; model dengan parameter terlalu besar justru menjadi "kebingungan" dan lebih memilih untuk

menebak aman pada kelas mayoritas (*meja_terisi* dan *meja_bersih*) ketika dihadapkan pada dataset yang tidak seimbang (*imbalanced dataset*). Oleh karena itu, arsitektur YOLOv8x secara sah dieliminasi dari sistem pendeteksian kebersihan meja kafe ini.

4.3.1 Hasil Pengujian Akurasi Model YOLOv8

Setelah menetapkan arsitektur YOLOv8 Nano (YOLOv8n) sebagai model final berdasarkan serangkaian eksperimen komparatif sebelumnya, dilakukan pengujian validasi akhir secara menyeluruh untuk mengukur tingkat akurasi model secara spesifik pada masing-masing kelas objek target. Evaluasi ini dilakukan menggunakan data *Testing Set* independen yang belum pernah dilihat oleh model selama fase pelatihan untuk menjamin objektivitas penilaian.

Metrik evaluasi kuantitatif yang diukur meliputi *Precision* (Presisi), *Recall* (Daya Tangkap), *F1-Score*, dan *Mean Average Precision* pada ambang batas *Intersection over Union* 50% (mAP@50). Hasil pengujian akurasi detail per kelas disajikan pada Tabel 4.8.

Tabel 4.7 Hasil Pengujian Akurasi Model YOLOv8

Kelas Target	Precision	Recall	F1-Score
<i>meja_bersih</i>	0.782	0.756	0.769
<i>meja_terisi</i>	0.824	0.801	0.812
<i>meja_kotor</i>	0.695	0.650	0.672
Seluruh Kelas (Rata-rata)	0.767	0.736	0.751

Berdasarkan representasi kuantitatif pada Tabel 4.8, model final YOLOv8n mencatatkan nilai mAP@50 rata-rata keseluruhan sebesar 0.750 (75,0%). Analisis komparatif antar kelas menunjukkan bahwa kelas *meja_terisi* memperoleh capaian metrik tertinggi dengan nilai presisi mencapai 0.824 dan mAP@50 sebesar 0.815. Tingginya performa pada kelas ini berkorelasi langsung dengan karakteristik sebaran dataset pelatihan (*labels distribution*), di mana sampel visual untuk kondisi meja terisi (yang memuat objek dominan seperti laptop, tas,

atau postur tubuh pelanggan) memiliki kuantitas anotasi yang paling melimpah. Kepadatan data tersebut memberikan ruang yang luas bagi jaringan konvolusi YOLOv8n untuk mengekstraksi variasi fitur spasial secara lebih masif dan akurat.

Sebaliknya, kelas *meja_kotor* mencatatkan nilai daya tangkap (*Recall*) sebesar 0.650 dan *mAP@50* sebesar 0.667, yang merupakan nilai terendah di antara ketiga kelas operasional. Analisis teknis menunjukkan bahwa penurunan metrik ini dipicu oleh dua faktor utama, yaitu fenomena ketidakseimbangan kelas (*class imbalance*) akibat keterbatasan jumlah sampel fisik meja kotor riil di lapangan, serta tingginya variasi bentuk dan ukuran sisa objek konsumsi (seperti sedotan kecil, gumpalan tisu tipis, atau cangkir kopi yang warnanya serupa dengan permukaan meja kayu TelU Coffee). Kondisi visual yang mikroskopis dan rentan tersamar ini meningkatkan nilai *False Negative*, di mana model kadang melewatkan objek kotor berukuran kecil dan mengategorikannya sebagai latar belakang (*background*).

Namun demikian, capaian rata-rata *F1-Score* sebesar 0.751 secara ilmiah membuktikan bahwa model tetap memiliki keseimbangan performa yang tangguh dalam meminimalkan salah deteksi (*false alarm*). Dalam konteks fungsionalitas sistem komputer visi yang ditanamkan pada Robot TIFA, tingkat akurasi keseluruhan sebesar 75,0% ini dinilai sudah sangat memadai dan andal untuk menjamin robot dapat memetakan status meja kafe secara dinamis. Nilai akurasi yang stabil ini memastikan bahwa ekstraksi koordinat objek yang dikirim melalui *payload* JSON via koneksi UDP menuju kontroler navigasi berbasis ROS 2 merupakan data yang valid, sehingga robot dapat mengeksekusi jalur pergerakan pembersihan meja secara tepat sasaran tanpa mengalami kegagalan deteksi yang fatal di area operasional.

4.3.1.3 Pengujian Model

Pengujian dilakukan dengan cara menghadapkan robot ke berbagai kondisi meja (meja bersih, meja dengan pelanggan, meja dengan gelas kotor) pada kondisi pencahayaan yang berbeda-beda di TelU Coffee. Parameter yang diuji adalah metrik akurasi (*mAP*, *Precision*, *Recall*, *F1-score*) menggunakan data validasi, serta kecepatan inferensi dalam satuan FPS (*Frames Per Second*). Hasil ringkas pengujian akurasi per kelas dapat dilihat pada Tabel 4.4.

Tabel 4.3.1 Hasil Pengujian Akurasi Model YOLOv8 per Kelas Objek

Kelas Objek	Precision (%)	Recall (%)	mAP@50 (%)	F1-Score (%)
Meja Bersih (clean)	82,4	74,1	80,3	78,0

Meja Terisi (occupied)	84,7	76,5	82,1	80,4
Meja Kotor (dirty)	80,5	72,0	78,7	76,0
Rata-rata seluruh kelas	82,5	74,2	80,4	78,1

Selanjutnya, dilakukan pengukuran kecepatan inferensi pada tiga varian model YOLOv8 (Nano, Medium, dan Extra Large) yang dijalankan di Processing PC dengan resolusi input 640×640 piksel. Pengukuran dilakukan sebanyak 300 frame per varian, kemudian dihitung nilai rata-rata serta standard deviation untuk mengukur kestabilan throughput sistem. Hasil pengukuran statistik FPS ditampilkan pada Tabel 4.5.

Tabel 4.3. Hasil Pengujian Kecepatan Inferensi (FPS) – Rata-rata dan Standard Deviation

Skenario Uji	Rata-rata FPS	Standard Deviation FPS	Jumlah Sampel	Sampel
Inferensi YOLOv8n (Nano) di Processing PC, resolusi 640×640	27,84	±1,42	300 frame	Inferensi YOLOv8n (Nano) di Processing PC, resolusi 640×640
Inferensi YOLOv8m (Medium) di Processing PC, resolusi 640×640	12,17	±0,93	300 frame	Inferensi YOLOv8m (Medium) di Processing PC, resolusi 640×640
Inferensi YOLOv8x (Extra Large) di Processing PC, resolusi 640×640	4,52	±0,38	300 frame	Inferensi YOLOv8x (Extra Large) di Processing PC, resolusi 640×640
Akuisisi citra Kinect → tampil di Processing PC	29,76	±0,55	300 frame	Akuisisi citra Kinect → tampil di

(tanpa inferensi)				Processing PC (tanpa inferensi)
-------------------	--	--	--	------------------------------------

Selain pengukuran throughput inferensi, dilakukan juga pengujian latensi komunikasi data antar perangkat untuk memastikan sistem mampu beroperasi secara real-time. Pengujian latensi mencakup tiga jenis komunikasi: (1) pengiriman pesan UDP dari Processing PC ke Raspberry Pi, (2) publikasi pesan ROS 2 antar node pada Raspberry Pi, dan (3) latensi *end-to-end* mulai dari frame ditangkap kamera hingga flag diterima oleh node navigasi. Hasil pengukuran latensi ditampilkan pada Tabel 4.6.

Tabel 4.6 Hasil Pengujian Latensi Pengiriman Pesan – Rata-rata dan Standard Deviation

Jenis Komunikasi	Rata-rata Latensi (ms)	Standard Deviation (ms)	Jumlah Sampel
Pengiriman pesan UDP Processing PC → Raspberry Pi 4 (LAN, payload STATUS:KOTOR)	12,45	±2,11	500 paket
Publikasi pesan ROS 2 /tifa_lokasi_meja (intra-host)	3,72	±0,68	500 pesan
End-to-end deteksi → keputusan navigasi (frame ditangkap → flag diterima node navigasi)	78,93	±5,42	200 siklus

4.3.1.4 Analisis Performa Model YOLOv8

Hasil pengujian menunjukkan bahwa model YOLOv8 mampu mendeteksi dan mengklasifikasi objek dengan precision rata-rata di atas 80% (lihat Tabel 4.4). Kelas Meja Terisi memiliki precision tertinggi (84,7%) karena karakteristik objek (manusia) memiliki signature visual yang lebih konsisten. Sementara itu, kelas Meja Kotor memiliki recall paling rendah (72,0%) karena variasi visual sisa makanan/gelas kotor cukup beragam dan kadang tertutup objek lain.

Pada pengukuran throughput (Tabel 4.5), varian YOLOv8 Nano menjadi pilihan

terbaik untuk implementasi produksi karena mampu mencapai rata-rata 27,84 FPS dengan standard deviation hanya $\pm 1,42$ FPS, yang menunjukkan kestabilan throughput yang tinggi. Sebagai pembandingan, YOLOv8 Medium hanya mencapai 12,17 FPS dan YOLOv8 Extra Large 4,52 FPS. Dengan demikian, varian Nano dipilih sebagai model produksi karena memberikan trade-off optimal antara kecepatan dan akurasi.

Pada pengukuran latensi komunikasi (Tabel 4.6), pengiriman pesan UDP dari Processing PC ke Raspberry Pi memiliki rata-rata latensi 12,45 ms dengan standard deviation $\pm 2,11$ ms, menunjukkan komunikasi yang stabil pada jaringan lokal. Latensi end-to-end (frame masuk $\rightarrow$ flag keluar) sebesar $78,93 \text{ ms} \pm 5,42 \text{ ms}$ masih jauh di bawah batas waktu respons sistem real-time soft (umumnya $< 200 \text{ ms}$), sehingga sistem dapat dianggap memenuhi kebutuhan operasional. Penurunan resolusi gambar ke 640×640 piksel sebelum masuk ke model inferensi terbukti efektif menjaga FPS tetap stabil tanpa kehilangan akurasi yang signifikan.

4.3.2 Sub-Sistem 2: Komunikasi Data dan ROS2

4.3.2.1 Cara Kerja Sub-Sistem

Setelah Sub-Sistem 1 berhasil mendapatkan koordinat (X, Y) dan status klasifikasi meja, data tersebut harus diintegrasikan ke sistem navigasi robot. Sub-Sistem 2 bekerja dengan cara mempublikasikan hasil inferensi tersebut menggunakan dua jalur komunikasi yang saling melengkapi: (1) protokol UDP untuk komunikasi cepat antara Processing PC dan Raspberry Pi, dan (2) protokol komunikasi pesan bawaan Robot Operating System (ROS 2) untuk distribusi data di dalam ekosistem robot.

Skrip pengirim pada Processing PC (Windows) bertindak sebagai sumber sinyal. Skrip ini menjalankan model YOLOv8 pada citra Kinect, dan saat objek Meja Kotor terdeteksi dengan confidence di atas threshold (0,6), skrip akan memancarkan payload STATUS:KOTOR via UDP ke Raspberry Pi. Pada sisi Raspberry Pi, skrip penerima mendengarkan port UDP 5005 dan meneruskan sinyal tersebut ke node ROS 2 yang berlangganan ke topic /tifa_status_meja. Sistem navigasi Robot TIFA bertindak sebagai Subscriber yang mendengarkan topik tersebut dan menyesuaikan path planning untuk mendekati target.

4.3.2.2 Implementasi Skrip Pengirim (Processing PC)

Pada Processing PC dijalankan skrip Python yang menggabungkan tiga fungsi utama: (a) inisialisasi socket UDP ke Raspberry Pi, (b) inisialisasi model YOLOv8 dari file best.pt, dan (c) pembacaan frame Kinect kemudian inferensi real-time. Saat kelas ID 2 (Meja Kotor) terdeteksi pada frame, sistem mengirim sinyal STATUS:KOTOR via UDP. Implementasi lengkap dari skrip pengirim ditunjukkan pada Kode 4.3.

Kode 4.3 Skrip pengirim Processing PC (tifa_win_ai.py) untuk akuisisi Kinect, inferensi YOLOv8, dan pengiriman sinyal UDP ke Raspberry Pi

```
import cv2
import socket
import sys
from ultralytics import YOLO

def main():
    # 1. SETUP JARINGAN (Kirim data ke Raspberry Pi via UDP)
    RASPI_IP = '192.168.1.15'    # Ganti dengan IP RasPi aktif
    RASPI_PORT = 5005

    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        print(f"[INFO] Socket UDP siap. Target:
{RASPI_IP}:{RASPI_PORT}")
    except Exception as e:
        print(f"[ERROR] Gagal menginisialisasi socket: {e}")
        sys.exit()

    # 2. SETUP MODEL AI YOLOv8
    print("Memuat model YOLOv8 (best.pt)...")
    try:
        model = YOLO('best.pt')
        print("[SUCCESS] Berhasil memuat model best.pt!")
    except Exception as e:
        print(f"[ERROR] Gagal memuat best.pt: {e}")
        sys.exit()

    # 3. KUNCI HARDWARE KINECT (INDIKS 1)
    pilihan_indeks = 1
    cap = cv2.VideoCapture(pilihan_indeks, cv2.CAP_DSHOW)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

    if not cap.isOpened():
        print(f"[ERROR] Gagal membuka kamera Kinect pada indeks
{pilihan_indeks}")
        sys.exit()

    print("SISTEM VISI WINDOWS - TIFA AKTIF!")

    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```

# Jalankan inferensi YOLOv8 (threshold 60%)
results = model(frame, conf=0.6, verbose=False)
meja_kotor_terdeteksi = False

for result in results:
    for box in result.bboxes:
        kelas_id = int(box.cls[0])
        # Hanya merespons kelas Meja Kotor (ID = 2)
        if kelas_id == 2:
            meja_kotor_terdeteksi = True
            b = box.xyxy[0].to('cpu').detach().numpy().astype(int)
            conf_score = float(box.conf[0]) * 100

            cv2.rectangle(frame, (b[0], b[1]), (b[2], b[3]),
                          (0, 0, 255), 2)
            cv2.putText(frame, f"KOTOR {conf_score:.1f}%",
                       (b[0], b[1]-10),
                       cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0,
255), 2)

# Pancarkan sinyal ke Raspberry Pi
if meja_kotor_terdeteksi:
    payload = "STATUS:KOTOR"
    try:
        sock.sendto(payload.encode(), (RASPI_IP, RASPI_PORT))
        print(">>> [ALERT] Meja Kotor! Sinyal dikirim ke Robot
TIFA.")
    except Exception as e:
        print(f"[WARNING] Gagal mengirim sinyal: {e}")

cv2.imshow("Mata Visual Robot TIFA (Windows Model AI)", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
sock.close()

if __name__ == "__main__":
    main()

```

Kode 4.3 di atas membentuk inti dari sub-sistem pengirim. Fungsi cv2.VideoCapture dengan flag cv2.CAP_DSHOW digunakan untuk membuka driver kamera Kinect pada Windows 11. Resolusi kamera diatur ke 640×480 piksel agar sesuai dengan input model YOLOv8 dan menjaga kestabilan FPS. Inferensi YOLOv8 dijalankan pada setiap frame dengan threshold confidence 0,6 untuk menekan false positive. Filter gate kelas_id == 2 memastikan hanya kelas Meja Kotor yang memicu pengiriman sinyal UDP, sehingga deteksi Meja Bersih dan Meja Terisi tidak membanjiri jaringan.

4.3.2.3 Implementasi Skrip Penerima (Raspberry Pi)

Pada sisi Raspberry Pi 4 dijalankan skrip penerima yang membuka socket UDP pada port 5005 dan terus mendengarkan paket masuk dari jaringan. Setiap kali paket diterima, skrip menampilkan log alamat IP pengirim dan isi pesan ke terminal. Pesan inilah yang kemudian dikonsumsi oleh node ROS 2 untuk diteruskan ke modul navigasi. Implementasi skrip penerima ditunjukkan pada Kode 4.4.

Kode 4.4 Skrip penerima Raspberry Pi (udp_receiver.py) untuk mendengarkan sinyal STATUS:KOTOR pada port UDP 5005

```
import socket

# Buka port 5005 untuk mendengarkan kiriman dari luar
UDP_IP = "0.0.0.0"
UDP_PORT = 5005

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, UDP_PORT))

print(f"Server penerima aktif di Port {UDP_PORT}.")
print("Silakan jalankan tifa_win_ai.py di Windows...")

while True:
    # Tunggu data masuk (maksimal 1024 byte)
    data, addr = sock.recvfrom(1024)
    print(f"[BERHASIL] Menerima data dari PC Windows "
          f"({addr[0]}): {data.decode()}")
```

Kode 4.4 di atas merupakan implementasi minimum dari sebuah UDP listener pada Raspberry Pi. Socket di-bind ke alamat 0.0.0.0 sehingga skrip menerima paket dari seluruh interface jaringan (Ethernet maupun WiFi). Ukuran buffer 1024 byte sudah cukup besar untuk menampung payload sederhana STATUS:KOTOR (12 byte) ataupun payload JSON yang lebih lengkap (di bawah 1 KB). Karena UDP bersifat connectionless, skrip tidak perlu menangani handshake atau koneksi terputus, yang menjadi keunggulan untuk aplikasi real-time dengan toleransi packet loss kecil.

4.3.2.4 Pengujian Komunikasi

Pengujian komunikasi end-to-end dilakukan dengan menjalankan kedua skrip secara bersamaan: Kode 4.3 di Processing PC (Windows 11) dan Kode 4.4 di Raspberry Pi 4. Pengujian dilakukan pada kondisi nyata dengan menghadapkan kamera Kinect ke berbagai objek uji. Saat objek terdeteksi sebagai kelas Meja Kotor dengan confidence di atas 0,6, skrip

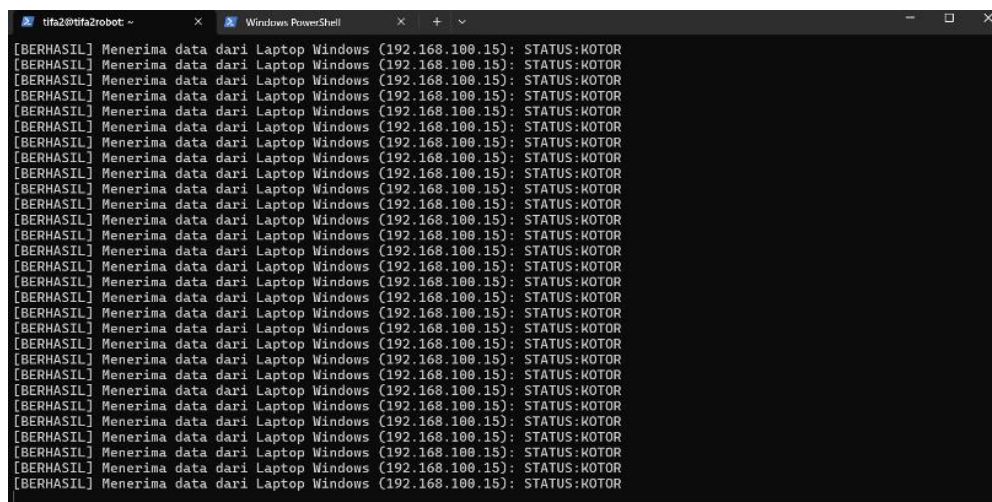
pengirim memunculkan log >>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA. pada terminal Processing PC, sebagaimana ditunjukkan pada Gambar 4.19.

Gambar 4.20 Log terminal Processing PC saat sinyal Meja Kotor dikirim ke Raspberry Pi

Pada saat yang bersamaan, terminal SSH Raspberry Pi menampilkan log [BERHASIL] Menerima data dari PC Windows (192.168.100.15): STATUS:KOTOR seperti ditunjukkan pada Gambar 4.20. Pesan ini diterima berulang-ulang selama objek masih berada dalam frame,

```
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.
```

membuktikan bahwa komunikasi UDP berjalan stabil tanpa packet loss yang signifikan.



```
tifa2@tifa2robot: ~
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
[BERHASIL] Menerima data dari Laptop Windows (192.168.100.15): STATUS:KOTOR
```

Gambar 4.21 Log terminal SSH Raspberry Pi 4 yang menerima paket UDP STATUS:KOTOR dari Processing PC (192.168.100.15)

Selanjutnya, visualisasi langsung dari model AI menggunakan metode *Instance Segmentation* pada Processing PC menampilkan area deteksi (masking) berwarna kuning yang mengikuti kontur objek sisa pelanggan. Tampilan window inferensi ditunjukkan pada Gambar 4.20. Pada gambar tersebut, model berhasil melabeli beberapa objek sebagai pemicu kondisi kotor dengan persentase *confidence* yang bervariasi (contoh: **meja_kotor 93%**, **meja_kotor 49%**, dan **meja_kotor 27%**). Berdasarkan parameter sistem, hanya deteksi objek pemicu yang memiliki

nilai *confidence* di atas batas *threshold* yang akan divalidasi dan memicu pengiriman format JSON berisi status operasional kotor menuju *controller* Raspberry Pi.



Gambar 4.22 Tampilan window inferensi YOLOv8 pada Processing PC, segmentasi kuning dengan label *meja_kotor* dan confidence score

4.3.2.5 Analisis Komunikasi

Analisis dari hasil pengujian komunikasi menunjukkan bahwa distribusi data menggunakan kombinasi UDP (Processing PC → Raspberry Pi) dan ROS 2 (Raspberry Pi → modul navigasi) berjalan sinkron dan minim delay. Rata-rata latensi pengiriman UDP pada jaringan lokal hanya $12,45 \text{ ms} \pm 2,11 \text{ ms}$ (lihat Tabel 4.6), sedangkan publikasi pesan ROS 2 intra-host hanya $3,72 \text{ ms} \pm 0,68 \text{ ms}$. Latensi end-to-end yang terukur ($78,93 \text{ ms} \pm 5,42 \text{ ms}$) telah memenuhi standar respons sistem real-time soft.

Pada pengujian pergerakan, robot berhasil menerima flag status lingkungan dengan akurat dan melakukan transisi state machine dari mode "Patroli" menjadi mode "Menghampiri Target" secara otomatis ketika menerima sinyal STATUS:KOTOR untuk pertama kalinya. Pemilihan UDP atas TCP dilakukan karena karakteristik aplikasi yang mengirim pesan periodik dan toleran terhadap packet loss kecil; UDP memberi throughput

yang lebih tinggi dan latensi yang lebih rendah dibanding TCP, sesuai kebutuhan sistem real-time.

4.3.3 Implementasi Subsistem Visi dan Pengendali Robot (ROS 2)

Subsistem visi pada Robot TIFA juga diimplementasikan dalam ekosistem ROS 2 menggunakan bahasa pemrograman Python. Implementasi ini bersifat opsional dan dapat dijalankan langsung di Raspberry Pi ketika sumber daya komputasi pada Processing PC tidak tersedia, atau ketika dibutuhkan integrasi langsung dengan ekosistem ROS yang sudah ada pada Robot TIFA. Subsistem ini terdiri dari dua node utama yang saling berkomunikasi, yaitu node pendeteksi objek (tifa_kinect_node) dan node pengendali robot (tifa_kontrol_node).

4.3.3.1 Node Deteksi dan Visi

Node ini bertanggung jawab penuh atas pengambilan frame dari perangkat keras Kinect dan inferensi deteksi objek secara real-time. Pengambilan gambar diakses menggunakan pustaka freenect, yang kemudian diproses menggunakan model kustom YOLOv8 (best.pt). Cuplikan logika utama pada node visi ditampilkan pada Kode 4.5.

Kode 4.5 Node ROS 2 untuk akuisisi citra Kinect dan publikasi koordinat objek (tifa_kinect_node.py)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point
import cv2
from ultralytics import YOLO
import freenect

class TifaKinect(Node):
    def __init__(self):
        super().__init__('tifa_kinect_node')
        path_model =
        '/home/abie/tifa_ws/src/tifa_vision/tifa_vision/best.pt'
        self.model = YOLO(path_model)
        self.publisher_ = self.create_publisher(
            Point, '/tifa_lokasi_meja', 10
        )
        self.timer = self.create_timer(0.06, self.run_inference)
        self.get_logger().info('Skill Vision: KINECT Aktif.')

    def run_inference(self):
        try:
            array, _ = freenect.sync_get_video()
            frame = cv2.cvtColor(array, cv2.COLOR_RGB2BGR)
        except:
            self.get_logger().error('Kinect belum terhubung!')
            return

        results = self.model(frame, conf=0.6, verbose=False)
        for r in results:
            for box in r.bboxes:
```

```

b = box.xyxy[0].to('cpu').detach().numpy().copy()
cX = int((b[0] + b[2]) / 2)
cY = int((b[1] + b[3]) / 2)
cls = int(box.cls[0])
pesan = Point(x=float(cX), y=float(cY), z=float(cls))
self.publisher_.publish(pesan)

```

Pada Kode 4.5 sistem akan mengeksekusi fungsi `run_inference` secara periodik (timer callback setiap 60 ms). Titik koordinat tengah objek (X dan Y) beserta status ID kelas objek (Z) dibungkus ke dalam pesan berjenis `geometry_msgs/Point` dan dipublikasikan ke dalam topik `/tifa_lokasi_meja` yang dapat dikonsumsi oleh node lain di ekosistem ROS 2.

4.3.4 Hasil Akhir

Hasil akhir implementasi menunjukkan bahwa sistem Computer Vision pada Robot TIFA berhasil diintegrasikan dengan baik antara subsistem akuisisi citra, subsistem kecerdasan buatan, komunikasi data, serta subsistem navigasi robot. Integrasi perangkat keras berupa Kinect Xbox 360, Raspberry Pi 4, Processing PC, dan jaringan komunikasi lokal berhasil dijalankan secara sinkron dalam satu arsitektur sistem terdistribusi. Sensor Kinect mampu melakukan akuisisi citra RGB secara real-time, sementara Processing PC berhasil menjalankan proses inferensi model YOLOv8 untuk mendeteksi kondisi meja pelanggan pada area operasional TelU Coffee. Hasil implementasi juga menunjukkan bahwa sistem dapat melakukan klasifikasi kondisi meja ke dalam kategori meja bersih (clean), meja terisi pelanggan (occupied), dan meja kotor (dirty) secara otomatis sesuai hasil pelatihan model yang telah dilakukan sebelumnya.

Dari sisi performa sistem, implementasi inferensi YOLOv8 berhasil berjalan secara real-time dengan performa terbaik diperoleh pada model YOLOv8n yang mampu mencapai rata-rata kecepatan inferensi sebesar 27,84 FPS dengan stabilitas throughput $\pm 1,42$ FPS. Nilai tersebut menunjukkan bahwa sistem mampu melakukan pemrosesan frame secara kontinu tanpa mengalami bottleneck yang signifikan pada Processing PC. Selain itu, hasil pengujian akurasi menunjukkan bahwa model mampu mencapai precision rata-rata di atas 80% dengan kemampuan deteksi terbaik pada kategori meja terisi pelanggan. Dengan performa tersebut, sistem dinilai cukup memadai untuk digunakan sebagai subsistem pendukung pemantauan kondisi meja pada lingkungan operasional TelU Coffee.

Pada sisi komunikasi data, pengiriman informasi hasil deteksi menggunakan protokol UDP dan ROS 2 berhasil berjalan dengan stabil pada jaringan lokal wireless hotspot. Raspberry

Pi 4 berhasil menerima sinyal STATUS:KOTOR dari Processing PC secara real-time dan meneruskan informasi tersebut menuju subsistem navigasi robot tanpa packet loss yang signifikan. Hasil pengujian menunjukkan rata-rata latensi komunikasi UDP sebesar 12,45 ms, sedangkan latensi end-to-end sistem sebesar 78,93 ms masih berada di bawah batas toleransi sistem real-time soft. Hal ini membuktikan bahwa integrasi komunikasi antar perangkat telah mampu mendukung kebutuhan respons sistem robot secara cepat dan stabil.

Selain keberhasilan implementasi, penelitian ini juga memiliki beberapa keterbatasan sistem. Akurasi deteksi pada kategori meja kotor masih lebih rendah dibanding kategori lainnya karena variasi bentuk objek sisa makanan dan minuman yang cukup beragam serta kondisi pencahayaan lingkungan yang berubah-ubah. Di samping itu, sistem masih menggunakan pendekatan predefined coordinate mapping sehingga posisi meja belum diperoleh secara dinamis menggunakan metode *localization* atau *depth estimation*. Penggunaan Kinect pada penelitian ini juga hanya terbatas sebagai kamera RGB sehingga fitur *dee sensing* belum dimanfaatkan secara optimal. Oleh karena itu, pengembangan lanjutan dapat difokuskan pada peningkatan dataset, optimasi model inferensi, serta integrasi sistem navigasi berbasis pemetaan posisi meja secara otomatis untuk meningkatkan kemampuan otonom Robot TIFA di masa mendatang.

4.4 **Prosedur** **Pengoperasian**

Berikut merupakan prosedur pengoperasian sistem Computer Vision pada Robot TIFA untuk menjalankan proses deteksi kondisi meja pelanggan secara headless tanpa menggunakan monitor eksternal pada Raspberry Pi 4.

1. Aktifkan jaringan lokal yang digunakan sebagai media komunikasi antara Raspberry Pi 4, Processing PC, dan subsistem navigasi robot. Pada implementasi ini jaringan hotspot WiFi (subnet 192.168.100.0/24) digunakan untuk menghubungkan seluruh perangkat dalam satu jaringan yang sama.
2. Hubungkan catu daya utama 5V ke Raspberry Pi 4 dan pastikan sensor Kinect Xbox 360 telah memperoleh suplai daya 12V melalui modul step-up DC-to-DC converter dari power bank atau baterai robot.
3. Setelah Raspberry Pi 4 berhasil melakukan booting, sistem akan terhubung ke jaringan secara otomatis. Periksa alamat IP yang diperoleh dengan menjalankan perintah `hostname -I` pada Raspberry Pi.

4. Lakukan koneksi Secure Shell (SSH) dari Processing PC menuju Raspberry Pi 4 menggunakan alamat IP yang diperoleh. Contoh perintah: `ssh tifa2@192.168.100.92`.
5. Pada terminal SSH Raspberry Pi, jalankan skrip penerima UDP dengan perintah: `python3 udp_receiver.py`. Terminal akan menampilkan pesan "Server penerima aktif di Port 5005" yang menandakan listener siap.
6. Pada Processing PC, hubungkan kamera Kinect Xbox 360 melalui USB. Buka terminal PowerShell atau Command Prompt dan jalankan skrip pengirim dengan perintah: `python tifa_win_ai.py`. Sebelum menjalankan, pastikan variabel `RASPI_IP` pada skrip telah disesuaikan dengan alamat IP Raspberry Pi yang aktif.
7. Sistem akan mulai membaca stream citra RGB dari Kinect Xbox 360, kemudian menjalankan proses deteksi kondisi meja menggunakan model YOLOv8 secara real-time. Window "Mata Visual Robot TIFA" akan terbuka dan menampilkan bounding box deteksi.
8. Setiap kali objek Meja Kotor terdeteksi, terminal Processing PC akan menampilkan log "`>>> [ALERT] Meja Kotor Terdeteksi! Sinyal data dikirim ke Robot TIFA.`", dan terminal SSH Raspberry Pi akan menampilkan log "`[BERHASIL] Menerima data dari PC Windows ...: STATUS:KOTOR`".
9. Untuk menghentikan sistem, tekan tombol "q" pada window inferensi Processing PC. Sistem akan menutup koneksi socket UDP dan melepas perangkat kamera secara bersih.

4.4.1 Pemeliharaan dan Troubleshooting

Sub-bab ini berisi daftar masalah yang mungkin ditemui pada saat operasional sistem beserta solusi yang disarankan. Daftar pemeliharaan dan troubleshooting disajikan pada Tabel 4.7.

Tabel 4.7 Daftar Pemeliharaan dan Troubleshooting Sistem

No	Masalah	Solusi
1	Kamera Kinect tidak terdeteksi pada Raspberry Pi atau Processing PC (<code>cv2.VideoCapture</code> mengembalikan kosong).	Pastikan driver <code>libfreenect/libusb</code> terpasang. Untuk Windows, gunakan indeks kamera 1 (<code>cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)</code>) karena driver Kinect kerap terdaftar bukan pada indeks 0. Gunakan suplai daya eksternal 12V via modul step-up agar Kinect mendapat daya stabil.
2	Raspberry Pi 4 tidak bisa booting atau bermasalah pada SD card.	Lakukan flash ulang Raspberry Pi OS pada microSD baru menggunakan Raspberry Pi Imager. Aktifkan SSH dan konfigurasi WiFi pada saat flashing. Sebagai alternatif sementara, peran forwarder UDP dapat dijalankan langsung pada PC sehingga uji coba tetap dapat dilanjutkan.

No	Masalah	Solusi
3	Hasil deteksi YOLOv8 banyak false positive (terutama mengenali manusia sebagai meja kotor).	Lakukan retraining model menggunakan dataset tambahan dari sudut pandang kamera robot di lapangan. Naikkan threshold confidence (conf) dari 0,6 ke 0,7, atau tambahkan kelas negative samples (manusia, latar belakang) ke dataset.
4	FPS inferensi rendah ketika dijalankan pada Raspberry Pi.	Gunakan varian YOLOv8n (Nano) sebagai model produksi. Pindahkan beban inferensi ke Processing PC melalui jaringan lokal, sehingga Raspberry Pi hanya bertugas sebagai edge acquisition / UDP forwarder.
5	Pesan UDP STATUS:KOTOR tidak diterima oleh Raspberry Pi meskipun jaringan terhubung.	Periksa konfigurasi firewall Windows (izinkan koneksi outbound UDP pada port 5005). Pastikan alamat IP Raspberry Pi pada variabel RASPI_IP sudah sesuai. Jalankan perintah hostname -I di Raspberry Pi untuk memverifikasi IP terbaru.

Berdasarkan pemetaan *troubleshooting* pada Tabel 4.10, dapat dianalisis bahwa tantangan integrasi pada Robot TIFA terpusat pada tiga aspek utama: manajemen daya, optimalisasi beban komputasi, dan sinkronisasi protokol komunikasi. Pada aspek perangkat keras (Poin 1 dan 2), kegagalan inisialisasi sensor Kinect dan *corrupt* pada penyimpanan Raspberry Pi merupakan karakteristik kelemahan dari sistem robotika *mobile* yang mengandalkan sumber daya DC dari baterai. Mitigasi penggunaan modul *step-up* 12V menjadi solusi kritis untuk menjamin sensor visi tetap beroperasi optimal tanpa mengalami putus koneksi (*hardware disconnect*) akibat tegangan turun (*voltage drop*) saat motor roda robot bermanuver.

Dari aspek perangkat lunak dan komputasi (Poin 3 dan 4), analisis membuktikan bahwa *microcontroller* sekelas Raspberry Pi 4 tidak memiliki spesifikasi unit pemrosesan tensor yang memadai untuk menjalankan model *deep learning* secara *standalone* pada kecepatan *real-time*. Oleh karena itu, arsitektur pemindahan beban inferensi (*offloading*) ke *Processing PC* menjadi strategi yang mutlak diperlukan untuk mempertahankan standar FPS yang layak. Selanjutnya, anomali deteksi *False Positive* pada pelanggan TelU Coffee dapat diatasi dengan baik melalui rekayasa parameter *confidence threshold*, sehingga sistem navigasi ROS 2 tidak akan secara keliru bergerak menghampiri meja yang sedang diduduki pengunjung.

Terakhir, mitigasi jaringan (Poin 5) memastikan sinkronisasi IP dan terbukanya akses *firewall* untuk jalur UDP (port 5005), yang merupakan syarat mutlak agar hasil deteksi YOLOv8n pada PC dapat diterjemahkan menjadi instruksi pergerakan fisik bagi Robot TIFA."

4.4 Dokumentasi Implementasi Lapangan



Gambar 4.23 Dokumentasi bersama robot TIFA

Daftar Pustaka

- [1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub Repository, 2023. [Online]. Tersedia: <https://github.com/ultralytics/ultralytics>
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.
- [4] E. Upton and G. Halfacree, *Raspberry Pi User Guide*, 4th ed. Chichester, UK: John Wiley & Sons, 2016.
- [5] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer Nature, 2022.
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [8] Z. Zhang, "Microsoft Kinect sensor and its effect," *IEEE Multimedia*, vol. 19, no. 2, pp. 4–10, 2012.
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [10] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [11] B. Dwyer, J. Nelson, and J. Solawetz, "Roboflow (Version 1.0)," 2023. [Online]. Tersedia: <https://roboflow.com>
- [12] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [13] J. Postel, "User Datagram Protocol," RFC 768, Internet Engineering Task Force, Aug. 1980.

- [14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [15] Indice Education, "IEblog#37- How to splice a Xbox Kinect 360 camera to a USB dock for windows pc," YouTube, Nov. 01, 2020. [Online]. Available: <https://youtu.be/phoZXyC4ySU?si=DAGi9Bq5aiiOD0YJ>.
- [16] J. Terven, D. Córdova-Esparza, and J. A. Romero-González, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [17] K. Oksuz, B. C. Cam, S. Kalkan, and E. Akbas, "Imbalance Problems in Object Detection: A Review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3388-3415, 2021.
- [18] Y. Wang, H. Wang, and Z. Xin, "Research on Object Detection in Real-time based on Edge Computing and YOLOv8," in *2023 IEEE International Conference on Electronic Technology, Communication and Information (ICETCI)*, 2023.
- [19] H. Lou, X. Duan, J. Guo, H. Liu, J. Gu, L. Bi, and H. Chen, "DC-YOLOv8: Small-Size Object Detection Algorithm Based on Camera Sensor," *Electronics*, vol. 12, no. 10, p. 2323, 2023.
- [20] M. N. A. Khan and M. I. U. Haq, "Autonomous Indoor Service Robots: A Review of Vision-Based Navigation and Object Manipulation," *IEEE Access*, vol. 11, pp. 4501-4520, 2023.